

Provable Cryptography for Bitcoin: An Introduction

Jonas Nick

Blockstream Research
jonas@n-ck.net

License: This work is released into the public domain under CC0 1.0
<https://creativecommons.org/publicdomain/zero/1.0/>

August 27, 2025

Table of Contents

1	Introduction	3
1.1	Algorithms	3
1.2	Negligible Functions	3
1.3	Games & Asymptotic Security	4
1.4	Hash Functions	5
1.5	Exercises	6
2	Reductions	7
2.1	Collision Resistance	8
2.2	Exercises	9
3	Commitments	9
3.1	Syntax	9
3.2	Security	9
3.3	Exercises	10
4	Accumulators	11
4.1	Syntax & Correctness	11
4.2	Security	12
4.3	Exercises	12
5	One-Time Signatures	13
5.1	Syntax & Correctness	13
5.2	Security	13
5.3	Lamport Signatures	14
5.4	Exercises	14
6	Discrete Logarithm	16
6.1	The Discrete Logarithm Problem	16
6.2	Pedersen Commitments	16
6.3	The DDH Assumption	17
6.4	ElGamal Commitments	17
6.5	The OMDL and AOMDL Problems	17
6.6	Exercises	18
7	Random Oracle Model	19
7.1	Random Oracles	19
7.2	ROM	20
7.3	Hash Commitments	21
7.4	Taproot Commitments	21
7.5	Exercises	21
8	Recap Quiz	22
9	Programmable ROM and Forking Lemma	23
9.1	Programmable Random Oracles	23
9.2	The Forking Lemma	24
9.3	Exercises	25
10	Signatures	26

10.1 Syntax & Correctness	26
10.2 Security	27
10.3 Schnorr Signatures	27
10.4 Exercises	29
11 Signatures with Key Tweaking	29
11.1 Syntax	29
11.2 Key Tweaking for Schnorr Signatures	30
11.3 Security with Tweaked Keys	30
11.4 Exercises	30
12 Interactive Aggregate Signatures	31
12.1 Exercises	31
A Notation	32
A.1 Sets and Numbers	32
A.2 Probability and Sampling	32
A.3 Cryptographic Notation	32
A.4 Groups and Fields	32
A.5 Logical Operators	32
A.6 Other Notation	32
B Appendix: Proofs of Negligible Function Properties	33
C Appendix: Notation	33
Changelog	33
References	33

1 Introduction

This workbook has two primary goals. First, it aims to provide sufficient background to understand state-of-the-art papers on cryptographic signatures, with a focus on discrete-logarithm-based multi-party signatures, including their security proofs. Second, it seeks to develop the skills needed to formalize security notions for cryptographic primitives. This skill is crucial for selecting appropriate primitives when proposing and reviewing cryptographic protocols, and for defining precisely what a protocol aims to achieve.

The concepts introduced in this section may at first seem abstract, but this level of abstraction is important: it provides the rigorous mathematical framework on which modern provable cryptography is built. Throughout this workbook we will study algorithms, analyse their running time and success probability, both for explicit algorithms and for hypothetical adversarial algorithms. In later sections we will sometimes study these algorithms within idealised computational models.

This workbook primarily contains definitions, propositions, and lemmas, with limited intuitive explanations. Good cryptography papers should include intuition, but it's never complete: intuition is inherently subjective and what makes sense to one reader may not resonate with another. What you need to do is develop your own explanations while reading the mathematics very precisely. The goal of this workbook is to build intuition step-by-step through the exercises, which provide hands-on experience with the theoretical concepts.

Note that there is rarely a single standard definition for a concept in cryptography. Sometimes we encounter equivalent definitions in the literature, while other times we find definitions that differ in subtle but important ways. This is why contemporary papers in cryptography include a preliminaries section that rigorously defines the concepts they use.

1.1 Algorithms

“In mathematics, *algorithm* is commonly understood to be an exact prescription, defining a computational process, leading from various initial data to the desired result.” (A. Markov, 1954)

Definition 1.1. *An algorithm $\mathcal{A}$ is*

- probabilistic *if it gets random bits as input (in addition to its regular input). We write $\mathcal{A}(x; \rho)$ to denote running algorithm $\mathcal{A}$ on input x with randomness ρ . When the randomness is clear from context or unnecessary, we simply write $\mathcal{A}(x)$.*
- polynomial-time *if there exists a polynomial p such that for every $x \in \{0, 1\}^\lambda$, $\mathcal{A}(x)$ terminates in time $p(\lambda)$.*
- probabilistic polynomial-time (p.p.t.) *if it is probabilistic and polynomial-time.*

Note that the runtime of an algorithm is characterized relative to the size of the input. All adversaries considered in cryptography are algorithms.

1.2 Negligible Functions

Definition 1.2. *A function $f : \mathbb{Z} \rightarrow \mathbb{R}$ is negligible if for every constant $c > 0$ there exists $N \in \mathbb{N}$ such that for all integers $\lambda > N$, $f(\lambda) < \lambda^{-c}$.*

Example 1. $2^{-\lambda}$ is negligible. $1/\lambda$ is not negligible.

If a function f is negligible in λ , we write $f(\lambda) = \text{negl}(\lambda)$.

Lemma 1.1 (Properties of negligible functions).

1. *Let f be a negligible function and p be a polynomial. Then $f(\lambda) \cdot p(\lambda)$ is negligible.*
2. *Let f_1, f_2 be negligible functions. Then $f_1 + f_2$ is negligible.*
3. *Let f be a negligible function $f(\lambda) \geq 0$ and $k > 0$ be a constant. Then $f + k$ is not negligible.*
4. *Let f be a non-negligible function and g be a negligible function. Then $f - g$ is not negligible.*

The proof of property (1) is left as an exercise (see [Exercise 1.6](#)). The proofs of properties (2)-(4) can be found in [Appendix B](#).

1.3 Games & Asymptotic Security

In this section we will introduce the concept of security games (sometimes called “experiments”) and asymptotic security through a series of toy examples.

The Game $\text{Guess}^{\mathcal{A}}(\lambda)$ defined in Figure 1.1 takes an algorithm $\mathcal{A}$ and the security parameter λ as input and outputs 1 or 0. The game first sets $n = 2^\lambda$ and samples x uniformly at random from $\mathbb{Z}_n$. Then it runs algorithm $\mathcal{A}$ without input, sets its output to x' and returns 1 if $x = x'$.

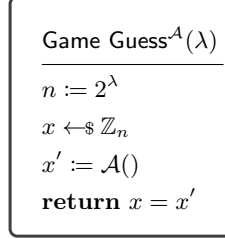


Fig. 1.1. Guessing game

Definition 1.3 (Guessing game advantage). For Game $\text{Guess}^{\mathcal{A}}(\lambda)$ as defined in Figure 1.1 we define the advantage of algorithm $\mathcal{A}$ as

$$\text{Adv}_{\mathcal{A}}^{\text{guess}}(\lambda) := \Pr \left[\text{Guess}^{\mathcal{A}}(\lambda) = 1 \right].$$

Proposition 1.1. For any algorithm $\mathcal{A}$, $\text{Adv}_{\mathcal{A}}^{\text{guess}}(\lambda) = \text{negl}(\lambda)$.

Proof. Let x' be the value returned by $\mathcal{A}$. The probability that x sampled independently and uniformly at random from $[1, 2^\lambda]$ is equal to x' is $2^{-\lambda} = \text{negl}(\lambda)$. $\square$

While the guessing game above provides a clean security definition, formalizing the security of concrete hash functions like SHA256 requires a fundamentally different approach. To illustrate this, consider the Game $\text{SHAPreimgZ}^{\mathcal{A}}(\lambda)$ defined in Figure 1.2, which outputs 1 if the algorithm $\mathcal{A}$ outputs a preimage of 0 under the hash function $\text{SHA256} : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$. Note that $\mathcal{A}$ gets 1^λ (a string of 1s of length λ) as input in order to characterise the running time of $\mathcal{A}$ as a function of the security parameter λ .

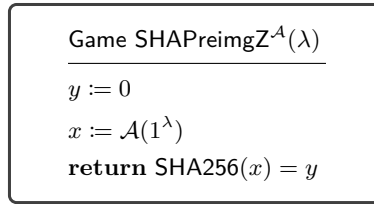


Fig. 1.2. Game for finding SHA256 preimage of zero

Proposition 1.2. For Game $\text{SHAPreimgZ}^{\mathcal{A}}(\lambda)$ as defined in Figure 1.2, let the advantage of $\mathcal{A}$ be defined as

$$\text{Adv}_{\mathcal{A}}^{\text{shapreimgZ}}(\lambda) := \Pr \left[\text{SHAPreimgZ}^{\mathcal{A}}(\lambda) = 1 \right].$$

If there exists x such that $\text{SHA256}(x) = 0$, there exists a p.p.t. algorithm $\mathcal{A}$ such that

$$\text{Adv}_{\mathcal{A}}^{\text{shapreimgZ}}(\lambda) = 1.$$

Proof. Let $\mathcal{A}$ be an algorithm

return x

where $\text{SHA256}(x) = 0$. □

Next, we consider a variant of preimage resistance for SHA256 where the target value is chosen uniformly at random, preventing the existence of trivial algorithms like the one in Proposition 1.2. In Game $\text{SHAPreimg}^{\mathcal{A}}(\lambda)$ defined in Figure 1.3 the value x is a uniformly random bitstring of length 256.

Game $\text{SHAPreimg}^{\mathcal{A}}(\lambda)$

$x \leftarrow \$ \{0, 1\}^{256}$
 $y := \text{SHA256}(x)$
 $x' := \mathcal{A}(y)$
return $\text{SHA256}(x') = y$

Fig. 1.3. Game for finding the SHA256 preimage of a random value

Proposition 1.3. For Game $\text{SHAPreimg}^{\mathcal{A}}(\lambda)$ as defined in Figure 1.3, let the advantage of $\mathcal{A}$ be defined as

$$\text{Adv}_{\mathcal{A}}^{\text{shaprei}}(\lambda) := \Pr \left[\text{SHAPreimg}^{\mathcal{A}}(\lambda) = 1 \right].$$

There exists a p.p.t. algorithm $\mathcal{A}$ such that

$$\text{Adv}_{\mathcal{A}}^{\text{shaprei}}(\lambda) = 1.$$

Proof. Let $\mathcal{A}$ be an algorithm that queries SHA256 until it finds a solution as shown in Figure 1.4. $\mathcal{A}$ will return the correct solution with probability 1. The running time of $\mathcal{A}$ is bounded by a

$\mathcal{A}(y)$

$x := 0$
while $\text{SHA256}(x) \neq y$ **do**
 $x := x + 1$
endwhile
return x

Fig. 1.4. Algorithm for finding the SHA256 preimage of a given value

constant independent of the security parameter λ and, therefore, $\mathcal{A}$ is p.p.t. □

We observe that if the output space of the hash function is independent of the security parameter, then the hash function is not preimage-resistant.

1.4 Hash Functions

Definition 1.4 (Hash function). A hash function H is a pair of polynomial-time algorithms $(\text{Gen}, \text{Eval})$ where:

- $\text{Gen}(1^\lambda) \rightarrow \kappa$ is a probabilistic algorithm that takes the security parameter as input and outputs a hashing key κ ¹.
- $\text{Eval}(\kappa, x) \rightarrow y$ is a deterministic algorithm that takes a hashing key κ and an input $x \in \{0, 1\}^*$ as input and outputs a hash value $y \in S$. When not specified otherwise, we assume $S = \{0, 1\}^\lambda$.²

Note that the hashing key κ is not secret because it is required to evaluate the hash function.

Remark 1.1 (Encoding of Arbitrary Inputs). We extend the notation to arbitrary inputs x (such as group elements, integers, or tuples) and simply write $\text{H.Eval}(\kappa, x)$, implicitly assuming the existence of a fixed injective encoding function that maps the input x into $\{0, 1\}^*$. We omit explicit reference to the encoding function in our notation.

Game Preimage_H^A(λ)

$\kappa := \text{H.Gen}(\lambda)$

$x \leftarrow_{\$} \{0, 1\}^\lambda$

$y := \text{H.Eval}(\kappa, x)$

$x' := \mathcal{A}(\kappa, y)$

return $\text{H.Eval}(\kappa, x') = y$

Fig. 1.5. Game for finding the preimage of a given value under the hash function

Similar to the algorithm in Figure 1.4, there exists an algorithm that wins $\text{Game Preimage}_H^A(\lambda)$ with probability 1 by just evaluating H.Eval on fresh inputs until $\text{H.Eval}(x) = y$. However, for adequate hash functions this algorithm runs in time exponential in the security parameter. Hence, the definition of preimage-resistance only considers p.p.t. adversaries.

Definition 1.5 (Preimage-resistance). A hash function H is preimage-resistant if for any p.p.t. algorithm $\mathcal{A}$,

$$\text{Adv}_{\mathcal{A}, H}^{\text{prei}}(\lambda) := \Pr \left[\text{Preimage}_H^A(\lambda) = 1 \right] = \text{negl}(\lambda).$$

Remark 1.2 (Asymptotic Security Framework). Definition 1.5 is an example of a definition in the asymptotic security framework. The hash function H consists of algorithms that are polynomial-time in λ , and preimage resistance requires that any p.p.t. adversary's success probability is negligible in λ . Hence, by increasing λ we can make the success probability of any p.p.t. adversary arbitrarily small.

1.5 Exercises

Exercise 1.1. Determine which of the following functions are negligible in λ :

1. $f(\lambda) = 0$
2. $f(\lambda) = \frac{1}{2}$
3. $f(\lambda) = \frac{1}{\lambda^2}$
4. $f(\lambda) = \lambda^{4096} \cdot 2^{-\lambda}$
5. $f(\lambda) = \sqrt{2^{-\lambda}}$

Exercise 1.2. Consider the $\text{Game Guess2}^A(\lambda)$ which is identical to $\text{Game Guess}^A(\lambda)$ except that x is sampled *after* $\mathcal{A}$ is run. That is, the game first runs $x' := \mathcal{A}()$, then samples $x \leftarrow_{\$} \mathbb{Z}_n$, and returns 1 if $x = x'$.

¹ We assume κ includes 1^λ so that Eval can run in time polynomial in λ

² For notational simplicity, $\text{H.Eval}(\kappa, x)$ is often written as $\text{H}(x)$, leaving the hashing key implicit.

1. Are these two games equivalent? That is, does $\text{Adv}_{\mathcal{A}}^{\text{guess}}(\lambda) = \text{Adv}_{\mathcal{A}}^{\text{guess}^2}(\lambda)$ for all algorithms $\mathcal{A}$?
2. Does this change affect the proof of the proposition above?

This exercise illustrates an important principle in game-based proofs: the order of operations does not necessarily affect game outcomes when the operations are independent.

Exercise 1.3. Why is the definition of $\text{Adv}_{\mathcal{A}}^{\text{shapreiz}}(\lambda)$ in Proposition 1.2 problematic from a cryptographic perspective? What fundamental issue does it illustrate about concrete hash functions like SHA256?

Exercise 1.4. Why is the definition of $\text{Adv}_{\mathcal{A}}^{\text{shaprei}}(\lambda)$ following Proposition 1.2 also problematic? Consider the adversary described in the proof.

Exercise 1.5. How does the definition of preimage resistance for hash functions (Definition 1.5) avoid the issues encountered with the SHA256 examples?

Exercise 1.6 (Optional). Prove property (1) of Lemma 1.1: If f is a negligible function and p is a polynomial, then $f(\lambda) \cdot p(\lambda)$ is negligible.

Hints:

- Any polynomial $p(\lambda)$ can be bounded by λ^a for some positive integer a and sufficiently large λ .
- To show $f(\lambda) \cdot p(\lambda)$ is negligible, for any $c > 0$, use the fact that f is negligible with constant $c + a$.

Exercise 1.7 (Optional). An alternative to the asymptotic approach is the *concrete security* approach. In this approach, we define (t, ϵ) -hardness as follows: The **Game Guess** ^{$\mathcal{A}$} (λ) as defined in Figure 1.1 is (t, ϵ) -hard if for any algorithm $\mathcal{A}$ running in time t , we have $\Pr[\text{Game Guess}^{\mathcal{A}}(\lambda) = 1] \leq \epsilon$.

Give a proposition about the concrete hardness of the guessing game, analogous to Proposition 1.1.

Exercise 1.8 (Optional). This question explores problems when defining security for hash functions. Read Section 1 and Section 4 of “Formalizing Human Ignorance” [Rog06].

1. What is the “Foundations-of-Hashing dilemma”?
2. To get around this, what alternative approach to defining security is suggested? What does “explicitly given” mean in this context?
3. (Optional) See also the discussion in Appendix B.7 and the proposed approaches in Appendix B.5 of [BL13].

2 Reductions

We use *reductions* to relate the hardness of winning various games. To show that problem Y is at least as hard as problem X , we construct a reduction: a p.p.t. algorithm that solves X using any algorithm that solves Y as a subroutine. This reduces problem X to Y : if there were an efficient algorithm that solves Y , we would also have an efficient algorithm that solves X . By contrapositive, if X is hard, then Y must also be hard.

We illustrate the concept of reductions through a series of examples.

Definition 2.1. For **Game PreimageEither**_H^A(λ) as defined in Figure 2.1 we define the advantage of $\mathcal{A}$ as

$$\text{Adv}_{\mathcal{A}, H}^{\text{preie}}(\lambda) := \Pr[\text{PreimageEither}_H^{\mathcal{A}}(\lambda) = 1].$$

Proposition 2.1. Let H be a hash function. If for all p.p.t. adversaries $\mathcal{A}$, it holds that

$$\text{Adv}_{\mathcal{A}, H}^{\text{preie}}(\lambda) = \text{negl}(\lambda)$$

then H is preimage-resistant.

Game PreimageEither_H^A(λ)

```

κ := H.Gen(λ)
x1, x2 ←  $\{0, 1\}^\lambda$ 
y1 := H.Eval(κ, x1)
y2 := H.Eval(κ, x2)
x :=  $\mathcal{A}(\kappa, y_1, y_2)$ 
return H.Eval(κ, x) = y1 ∨ H.Eval(κ, x) = y2

```

Fig. 2.1. Game for finding the preimage of either of two given values under the hash function

$\mathcal{B}_H^A(\kappa, y_1, y_2)$

```

x :=  $\mathcal{A}(\kappa, y_1)$ 
return x

```

Fig. 2.2. Algorithm for finding the preimage of either of two given values under the hash function

Proof. We prove the contrapositive statement (which is equivalent to the statement in the proposition): If H is not preimage-resistant, then there exists a p.p.t. algorithm $\mathcal{B}$ that wins

Game PreimageEither_H^B(λ) with non-negligible probability.

If H is not preimage-resistant, then there exists a p.p.t. algorithm $\mathcal{A}$ that wins Game Preimage_H^A(λ) with non-negligible probability. Let $\mathcal{B}_H^A$ be the algorithm defined in Figure 2.2. It gets both challenges y_1, y_2 , runs $\mathcal{A}$ on y_1 and returns its output.

The success probability of $\mathcal{B}$ is at least that of $\mathcal{A}$. When $\mathcal{A}$ successfully finds a preimage of y_1 , then $\mathcal{B}$ succeeds in Game PreimageEither. Additionally, there is a non-zero probability that $H.Eval(\kappa, x) = y_2$ even when $H.Eval(\kappa, x) \neq y_1$. Therefore, $\text{Adv}_{\mathcal{B}_H^A, H}^{\text{preie}}(\lambda) \geq \text{Adv}_{\mathcal{A}, H}^{\text{prei}}(\lambda)$. Since $\text{Adv}_{\mathcal{A}, H}^{\text{prei}}(\lambda)$ is non-negligible and $\mathcal{B}_H^A$ is p.p.t., we have found a p.p.t. algorithm with non-negligible advantage for Game PreimageEither. $\square$

Proposition 2.2. *Let H be a preimage-resistant hash function. Then $\text{Adv}_{\mathcal{A}, H}^{\text{preie}}(\lambda)$ is negligible for all p.p.t. adversaries $\mathcal{A}$. More precisely, for any p.p.t. adversary $\mathcal{A}$ against Game PreimageEither of H , there exists a p.p.t. adversary $\mathcal{B}$ against the preimage-resistance of H such that*

$$\text{Adv}_{\mathcal{A}, H}^{\text{preie}}(\lambda) \leq 2\text{Adv}_{\mathcal{B}_H^A, H}^{\text{prei}}(\lambda).$$

The proof is left as an exercise (see Exercise 2.1).

2.1 Collision Resistance

Game Collision_H^A(λ)

```

κ := H.Gen(λ)
(x, x') :=  $\mathcal{A}(\kappa)$ 
return (x ≠ x' ∧ H.Eval(κ, x) = H.Eval(κ, x'))

```

Fig. 2.3. Game for finding a collision under the hash function

Definition 2.2 (Collision-resistance). Hash function H is collision-resistant if for any p.p.t. algorithm A ,

$$\text{Adv}_{A,H}^{\text{coll}}(\lambda) := \Pr \left[\text{Collision}_H^A(\lambda) = 1 \right] = \text{negl}(\lambda).$$

Theorem 2.1 (Collision-resistance implies preimage-resistance). Let H be a collision-resistant hash function. Then H is preimage-resistant. More precisely, for any p.p.t. adversary A against Game Preimage of H , there exists a p.p.t. adversary B against Game Collision of H such that

$$\text{Adv}_{A,H}^{\text{prei}}(\lambda) \leq \text{Adv}_{B,H}^{\text{coll}}(\lambda) + 2^{-\lambda}.$$

The proof is left as an exercise (see [Exercise 2.2](#)).

2.2 Exercises

Exercise 2.1. Prove [Proposition 2.2](#).

Hint: Construct a reduction that randomly places the challenge in either the first or second position.

Exercise 2.2. Prove [Theorem 2.1](#) (collision-resistance implies preimage-resistance).

Hint: Construct a reduction that samples a random x , computes $y = H.\text{Eval}(\kappa, x)$, and uses the preimage-finding adversary to find x' such that $H.\text{Eval}(\kappa, x') = y$. What is the probability that $x = x'$?

Exercise 2.3 (Optional). Describe target collision resistance, extended target collision resistance, and multi-target collision resistance [\[HRS16\]](#) in your own words and discuss their security against classical and quantum attacks (see Table 1 in the referenced paper).

Exercise 2.4 (Optional). Drijvers et al. [\[Dri+19\]](#) give a *metareduction* showing that MuSig(1) [\[Max+19\]](#) without the first nonce-commitment round cannot be proven secure under the one-more discrete logarithm (OMDL) assumption. Interpret Theorem 1 and Figure 2 in their paper. Complete the following informal statement: If there exists an algorithm B that reduces n -OMDL to the EUF-CMA security of the MuSig(1) variant, then there exists a reduction M and a forger F that solve the _____ problem.

3 Commitments

3.1 Syntax

Definition 3.1 (Commitment Scheme). A commitment scheme Com is a pair of polynomial-time algorithms $(\text{Setup}, \text{Commit})$ where:

- $\text{Setup}(1^\lambda) \rightarrow pp^3$ is a probabilistic algorithm that takes the security parameter as input and outputs public parameters pp including the message space $\mathcal{M}$ and the randomness space $\mathcal{R}$.
- $\text{Commit}(pp, m, r) \rightarrow C^4$ is a deterministic algorithm that takes public parameters pp , a message $m \in \mathcal{M}$, and randomness $r \in \mathcal{R}$ as input and outputs a commitment C .

Example 2. An example is the toy commitment scheme:

- $\text{Setup}(1^\lambda) := (\mathcal{M} := \{0, 1\}^*, \mathcal{R} := \{0\})$
- $\text{Commit}(pp, m, r) := m$

3.2 Security

Definition 3.2 (Binding). A commitment scheme Com is binding if for any p.p.t. algorithm A ,

$$\text{Adv}_{A,\text{Com}}^{\text{bind}}(\lambda) := \Pr \left[\text{ComBind}_{\text{Com}}^A(\lambda) = 1 \right] = \text{negl}(\lambda).$$

If $\text{Adv}_{A,\text{Com}}^{\text{bind}}(\lambda) = 0$, then the commitment scheme is said to be perfectly binding.

```

Game ComBindComA(λ)
-----
pp ←$ Setup(1λ)
(m0, m1, r0, r1) := A(pp)
C0 := Commit(pp, m0, r0)
C1 := Commit(pp, m1, r1)
return (C0 = C1) ∧ (m0 ≠ m1)

```

Fig. 3.1. Binding game

```

Game ComHideComA(λ)
-----
pp ←$ Setup(1λ)
(m0, m1) := A(pp)
r ←$ pp.ℛ
b ←$ {0, 1}
C := Commit(pp, mb, r)
b' := A(pp, C)
return (b = b')

```

Fig. 3.2. Hiding game

Definition 3.3 (Hiding). A commitment scheme Com is hiding if for any p.p.t. algorithm $\mathcal{A}$,

$$\text{Adv}_{\mathcal{A}, \text{Com}}^{\text{hide}}(\lambda) := |\Pr [\text{ComHide}_{\text{Com}}^{\mathcal{A}}(\lambda) = 1] - \frac{1}{2}| = \text{negl}(\lambda).$$

If $\text{Adv}_{\mathcal{A}, \text{Com}}^{\text{hide}}(\lambda) = 0$, then the commitment scheme is said to be perfectly hiding.

Note that in $\text{ComHide}_{\text{Com}}^{\mathcal{A}}(\lambda)$ the adversary $\mathcal{A}$ is run twice with different arguments, first to output two messages and then to output a bit.

3.3 Exercises

Exercise 3.1. Is the toy commitment scheme in [Example 2](#) binding? Prove it.

Exercise 3.2. A commitment scheme is *strongly binding* if it binds to *both* the message and randomness. Formally define the strong binding game $\text{ComStrongBind}_{\text{Com}}^{\mathcal{A}}(\lambda)$ by modifying $\text{ComBind}_{\text{Com}}^{\mathcal{A}}(\lambda)$.

Exercise 3.3 (Optional). Describe a scenario where strong binding is important.

Exercise 3.4. Design a hash-based commitment scheme $\text{HashCom}[\text{H}]$ ⁵ that is strongly binding when used with a collision-resistant hash function H . State and prove the security theorem.

Note: We do not yet have the tools to prove that $\text{HashCom}[\text{H}]$ is hiding; we will return to this in later chapters.

Exercise 3.5. Is the toy commitment scheme in [Example 2](#) hiding? Prove it.

³ The security parameter 1^λ is often left implicit in algorithm inputs to simplify notation, e.g., writing $\text{Setup}()$ instead of $\text{Setup}(1^\lambda)$.

⁴ Public parameters pp are often left implicit in algorithm inputs to simplify notation, e.g., writing $\text{Commit}(m, r)$ instead of $\text{Commit}(pp, m, r)$.

⁵ The notation $\text{Scheme}[\text{Primitive}]$ indicates that the scheme is parameterized by a cryptographic primitive.

Exercise 3.6. Let H be a collision-resistant hash function and define the commitment scheme $\text{HashCom}'[H]$ as:

- $\text{Setup}(1^\lambda) := (\mathcal{M} := \{0, 1\}, \mathcal{R} := \{0\}, \kappa := H.\text{Gen}(1^\lambda))$
- $\text{Commit}(pp, m, r) := H.\text{Eval}(\kappa, m)$

Note that $\text{HashCom}'[H]$ is binding. Is it hiding? Prove it.

4 Accumulators

4.1 Syntax & Correctness

An accumulator is a compact representation of a set of elements and allows certain operations on the set.

Definition 4.1 (Accumulator). *An accumulator that supports insertions of elements and membership proofs is also called an additive positive accumulator [BCY20]. It is a tuple of polynomial-time algorithms $(\text{Gen}, \text{Init}, \text{Value}, \text{Insert}, \text{ProveMembership}, \text{VerifyMembership})$ where:*

- $\text{Gen}(1^\lambda) \rightarrow pp$ is a probabilistic algorithm that takes the security parameter as input and outputs public parameters pp including the domain $\mathcal{X}_{\text{Acc}}$ of elements that can be accumulated.
- $\text{Init}(pp) \rightarrow \text{state}$ is a deterministic algorithm that takes public parameters pp as input and outputs the initial state, state , of the accumulator.
- $\text{Value}(pp, \text{state}) \rightarrow v$ is a deterministic algorithm that takes public parameters pp and an accumulator state, state , as input and outputs the accumulator value v .
- $\text{Insert}(pp, \text{state}, x) \rightarrow \text{state}'$ is a deterministic algorithm that takes public parameters pp , an accumulator state, state , and an element $x \in \mathcal{X}_{\text{Acc}}$ as input and outputs a new state, state' , after inserting x .
- $\text{ProveMembership}(pp, \text{state}, x) \rightarrow \pi$ is a probabilistic algorithm that takes public parameters pp , an accumulator state, state , and an element x as input and outputs a membership proof π .
- $\text{VerifyMembership}(pp, v, x, \pi) \rightarrow \{0, 1\}$ is a deterministic algorithm that takes public parameters pp , an accumulator value v , an element $x \in \mathcal{X}_{\text{Acc}}$, and a proof π as input and outputs 1 (accept) if the proof validates that x is a member of the set represented by v , and outputs 0 (reject) otherwise.

Typically, an accumulator involves two parties: the manager, who uses Init , Value , Insert , ProveMembership and the verifier, who uses VerifyMembership .

Example 3. An example is the trivial accumulator, where the accumulator value is simply the set itself and therefore grows linearly with the number of inserted elements. (In contrast, non-trivial accumulators typically have constant-size accumulator values.)

- $\text{Gen}(1^\lambda) \rightarrow pp$ where $pp = (\mathcal{X}_{\text{Acc}} = \{0, 1\}^*)$
- $\text{Init}(pp) \rightarrow \{\}$
- $\text{Value}(pp, \text{state}) \rightarrow \text{state}$
- $\text{Insert}(pp, \text{state}, x) \rightarrow \text{state} \cup \{x\}$
- $\text{ProveMembership}(pp, \text{state}, x) \rightarrow \perp$
- $\text{VerifyMembership}(pp, v, x, \pi) \rightarrow b$ outputs $b = 1$ if $x \in v$. Otherwise, it outputs $b = 0$.

Definition 4.2 (Correctness). *An additive positive accumulator is considered correct if, for any element that was inserted into the accumulator, the membership proof generated honestly for it is successfully verified. More formally, for all security parameters λ , all public parameters $pp := \text{Gen}(1^\lambda)$, all positive integers t polynomial in λ , all tuples $\vec{y} \in \mathcal{X}_{\text{Acc}}^t$: for all $x \in \vec{y}$,*

$$\Pr \left[\begin{array}{l} \text{state}_0 := \text{Init}(pp); \\ \text{state}_i := \text{Insert}(pp, \text{state}_{i-1}, y_i) \text{ for } i \in [1, t]; \\ \pi := \text{ProveMembership}(pp, \text{state}_t, x); \\ v := \text{Value}(pp, \text{state}_t); \\ \text{VerifyMembership}(pp, v, x, \pi) = 1 \end{array} \right] = 1 - \text{negl}(\lambda).$$

4.2 Security

In Game $\text{AccColl}^{\mathcal{A}}(\lambda)$ (defined in Figure 4.1), the adversary $\mathcal{A}$ is given access to *oracles*, which he can call, but not look inside. In other words, the adversary can insert elements into the accumulator and obtain membership proofs, but does not have access to the state directly.

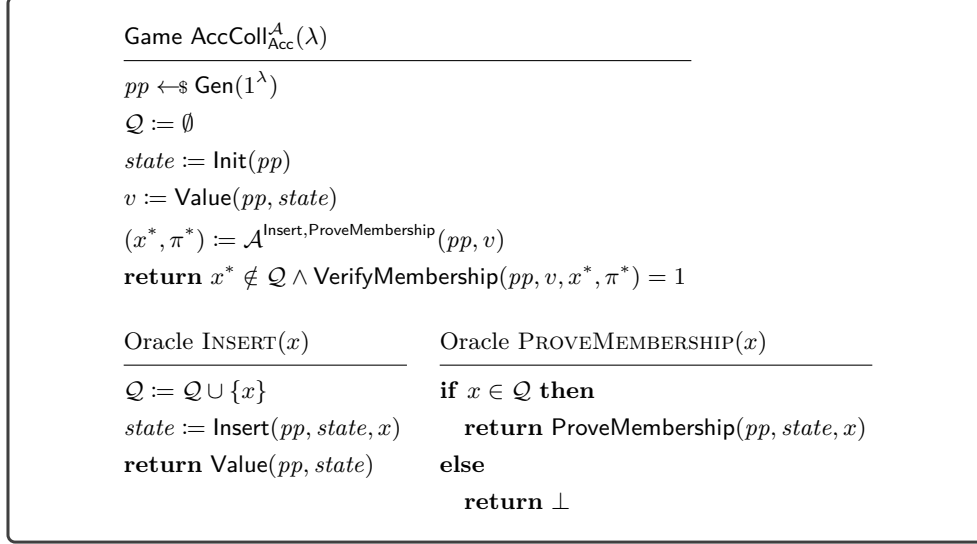


Fig. 4.1. Collision game

Definition 4.3 (Collision-Freeness). An additive positive accumulator is collision-free if it is hard to fabricate a membership proof for a value that is not in the set. More formally, an accumulator Acc is collision-free if for any p.p.t. algorithm $\mathcal{A}$

$$\text{Adv}_{\mathcal{A}, \text{Acc}}^{\text{collision}}(\lambda) := \Pr \left[\text{AccColl}_{\text{Acc}}^{\mathcal{A}}(\lambda) = 1 \right] = \text{negl}(\lambda).$$

4.3 Exercises

Exercise 4.1. Is the trivial accumulator correct? Justify or prove it.

Exercise 4.2. Is the trivial accumulator collision-free? Justify or prove it.

Exercise 4.3. Using a collision-resistant hash function, give an example of an additive positive accumulator with constant-size accumulator values. Argue that it is correct and collision-free.

Exercise 4.4 (Optional). Using a collision-resistant hash function, give an example of an additive positive accumulator with constant-size accumulator values and logarithmic-size membership proofs. Argue that it is correct and collision-free.

Exercise 4.5. Define the syntax, correctness, and collision-freeness of an additive universal accumulator (i.e., an additive positive accumulator that supports non-membership proofs).

Exercise 4.6 (Optional). Using a collision-resistant hash function, give an example of an additive universal accumulator with constant-size accumulator values and logarithmic-size membership and non-membership proofs. Argue that it is correct and collision-free.

Exercise 4.7 (Optional). Read Section 2 of Baldimtsi, Canetti, and Yakoubov [BCY20]. Describe the notion of *Strength* for accumulators.

Exercise 4.8 (Optional). Read Sections 3.5, 3.6, and Appendix A.2 of Shielded CSV [NEL25].

1. How do the accumulators required for Shielded CSV differ from the accumulators discussed in this section?
2. How would you approach proving the ToSA-SEC security property (an open problem at the time of writing)?

Exercise 4.9 (Optional). Read Section 5 of Cremer, Loss, and Wagner [CLW24], which provides security notions for various cryptographic primitives. Summarize the key insights.

5 One-Time Signatures

5.1 Syntax & Correctness

Definition 5.1 (One-Time Signature). A one-time signature scheme OTS is a tuple of polynomial-time algorithms $(\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$ where:

- $\text{Setup}(1^\lambda) \rightarrow pp$ is a probabilistic algorithm that takes the security parameter as input and outputs public parameters pp including the message space $\mathcal{M}$.
- $\text{KeyGen}(pp) \rightarrow (sk, pk)$ is a probabilistic algorithm that takes public parameters pp as input and outputs a secret key sk and a public key pk .
- $\text{Sign}(pp, sk, m) \rightarrow \sigma$ is a probabilistic algorithm that takes public parameters pp , a secret key sk , and a message $m \in \mathcal{M}$ as input and outputs a signature σ .
- $\text{Verify}(pp, pk, m, \sigma) \rightarrow \{0, 1\}$ is a deterministic algorithm that takes public parameters pp , a public key pk , a message $m \in \mathcal{M}$, and a signature σ as input and outputs 1 (accept) if the signature is valid, and outputs 0 (reject) otherwise.

Definition 5.2 (Correctness). A one-time signature scheme $\text{OTS} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$ is correct if for all $\lambda \in \mathbb{N}$, all $pp \in \text{Setup}(1^\lambda)$, all $(sk, pk) \in \text{KeyGen}(pp)$, and all messages $m \in \mathcal{M}$:

$$\Pr[\text{Verify}(pp, pk, m, \text{Sign}(pp, sk, m)) = 1] = 1$$

where the probability is taken over the randomness of Sign .

5.2 Security

Definition 5.3 (EUF-CMA Security). A one-time signature scheme OTS is EUF-CMA secure (Existential Unforgeability under Chosen Message Attack) if for all p.p.t. adversaries $\mathcal{A}$, it holds that

$$\text{Adv}_{\mathcal{A}, \text{OTS}}^{\text{euf-cma}}(\lambda) := \Pr \left[\text{Game EUF-CMA}_{\text{OTS}}^{\mathcal{A}}(\lambda) = 1 \right] = \text{negl}(\lambda).$$

Remark 5.1 (Assert notation). Given a predicate P , we use “**assert** P ” as a shorthand for “**if** $\neg P$ **then return** 0” in the main procedure or “**if** $\neg P$ **then return** $\perp$ ” in an oracle procedure.

Game $\text{EUF-CMA}_{\text{OTS}}^{\mathcal{A}}(\lambda)$	Oracle $\text{SIGN}(m)$
$pp := \text{Setup}(1^\lambda)$	assert $\mathcal{Q} = \emptyset$
$(sk, pk) := \text{KeyGen}(pp)$	$\mathcal{Q} := \{m\}$
$\mathcal{Q} := \emptyset$	return $\text{Sign}(pp, sk, m)$
$(m^*, \sigma) := \mathcal{A}^{\text{SIGN}}(pp, pk)$	
return $m^* \notin \mathcal{Q} \wedge \text{Verify}(pp, pk, m^*, \sigma) = 1$	

Fig. 5.1. The EUF-CMA security game for a one-time signature scheme.

Setup(1^λ) $pp := \text{H.Gen}(1^\lambda)$ return pp	Sign(pp, sk, m) // Sign must be called at most once per sk. $(m_1, \dots, m_k) := m$ $(sk_{1,0}, sk_{1,1}), \dots, (sk_{k,0}, sk_{k,1}) := sk$ $\sigma := (sk_{1,m_1}, \dots, sk_{k,m_k})$ return σ
KeyGen(pp) $x_{i,j} \leftarrow \$ \{0, 1\}^\lambda$ for $i \in [1, k], j \in \{0, 1\}$ $sk := (x_{1,0}, x_{1,1}, \dots, x_{k,0}, x_{k,1})$ $pk := (\text{H.Eval}(pp, x_{1,0}), \text{H.Eval}(pp, x_{1,1}), \dots,$ $\quad \text{H.Eval}(pp, x_{k,0}), \text{H.Eval}(pp, x_{k,1}))$ return (sk, pk)	Verify(pp, pk, m, σ) $(m_1, \dots, m_k) := m$ $(pk_{1,0}, pk_{1,1}), \dots, (pk_{k,0}, pk_{k,1}) := pk$ return $\bigwedge_{i=1}^k \text{H.Eval}(pp, \sigma_i) = pk_{i,m_i}$

Fig. 5.2. The one-time Lamport signature scheme $\text{LS}[\text{H}, k]$.

5.3 Lamport Signatures

Definition 5.4 (Lamport Signature Scheme). Let H be a hash function and k be a positive integer. The Lamport one-time signature scheme $\text{LS}[\text{H}, k]$ with message space $\mathcal{M} = \{0, 1\}^k$ is defined in [Figure 5.2](#).

Proposition 5.1 (Correctness of Lamport Signatures). The Lamport signature scheme $\text{LS}[\text{H}, k]$ is correct.

The proof of [Proposition 5.1](#) is left as an exercise (see [Exercise 5.4](#)).

Theorem 5.1 (Security of Lamport Signatures). Let H be a preimage-resistant hash function. Then the Lamport one-time signature scheme $\text{LS}[\text{H}, k]$ is EUF-CMA secure. More precisely, for any p.p.t. adversary $\mathcal{A}$ against the EUF-CMA security of $\text{LS}[\text{H}, k]$, there exists a p.p.t. adversary $\mathcal{B}$ against the preimage resistance of H such that

$$\text{Adv}_{\mathcal{A}, \text{LS}[\text{H}, k]}^{\text{euf-cma}}(\lambda) \leq 2k \text{Adv}_{\mathcal{B}, \text{H}}^{\text{preimage}}(\lambda).$$

Proof. [Figure 5.3](#) shows a sequence of games for the proof that Lamport signatures are EUF-CMA secure. The first game is identical to the **Game EUF-CMA** game. Each subsequent game introduces a small change compared to the previous game. Then we can construct a p.p.t. algorithm that runs a **Game EUF-CMA** adversary $\mathcal{A}$ internally and wins **Game Preimage** $_{\text{H}}^{\mathcal{A}}(\lambda)$ with the same probability as $\mathcal{A}$ winning the last **Game G**₂. The rest of the proof is left as an exercise (see [Exercise 5.5](#)). $\square$

5.4 Exercises

Exercise 5.1. What is “existential unforgeability” in “EUF-CMA” ([Definition 5.3](#)) and what is “chosen message attack”?

Exercise 5.2. To see why the security definition for signatures includes a signing oracle, consider **Game G** $_{\text{OTS}}^{\mathcal{A}}(\lambda)$ which is the same as **Game EUF-CMA** $_{\text{OTS}}^{\mathcal{A}}(\lambda)$ except that the adversary $\mathcal{A}$ is not given access to the signing oracle. Give an example of a one-time signature scheme that is correct and achieves $\Pr[\text{Game G}_{\text{OTS}}^{\mathcal{A}}(\lambda) = 1] = \text{negl}(\lambda)$ for all p.p.t. adversaries $\mathcal{A}$ but is not EUF-CMA secure.

Exercise 5.3. Is $(sk_{1,0}, sk_{2,0}, sk_{3,1})$ a valid signature for the message $m = (0, 0, 1)$ under the Lamport signature scheme $\text{LS}[\text{H}, 3]$?

Game $\text{EUF-CMA}_{\text{LS}[\text{H},k]}^{\mathcal{A}}(\lambda)$	Game $\text{G}_1^{\mathcal{A}}(\lambda)$	Game $\text{G}_{2\text{OTS}}^{\mathcal{A}}(\lambda)$
$pp := \text{Setup}(1^\lambda)$	$pp := \text{Setup}(1^\lambda)$	$pp := \text{Setup}(1^\lambda)$
$(sk, pk) := \text{KeyGen}(pp)$	$(sk, pk) := \text{KeyGen}(pp)$	$(sk, pk) := \text{KeyGen}(pp)$
$\mathcal{Q} := \emptyset$	$i \leftarrow \$ [1, k]; j \leftarrow \$ \{0, 1\}$	$i \leftarrow \$ [1, k]$
$(m^*, \sigma) := \mathcal{A}^{\text{SIGN}}(pp, pk)$	$\mathcal{Q} := \emptyset$	$j \leftarrow \$ [0, 1]$
return $m^* \notin \mathcal{Q} \wedge \text{Verify}(pp, pk, m^*, \sigma) = 1$	$(m^*, \sigma) := \mathcal{A}^{\text{SIGN}}(pp, pk)$	$\mathcal{Q} := \emptyset$
Oracle $\text{SIGN}(m)$	if $\mathcal{Q} = \emptyset$ then	$(m^*, \sigma) := \mathcal{A}^{\text{SIGN}}(pp, pk)$
assert $\mathcal{Q} = \emptyset$	$m \leftarrow \$ \{0, 1\}^k$	if $\mathcal{Q} = \emptyset$ then
$\mathcal{Q} := \{m\}$	$\text{SIGN}(m)$	$m \leftarrow \$ \{0, 1\}^k$
return $\text{Sign}(pp, sk, m)$	return $m^* \notin \mathcal{Q} \wedge \text{Verify}(pp, pk, m^*, \sigma) = 1$	$\text{SIGN}(m)$
	Oracle $\text{SIGN}(m)$	assert $m_i^* = j$
	assert $\mathcal{Q} = \emptyset$	return $m^* \notin \mathcal{Q} \wedge \text{Verify}(pp, pk, m^*, \sigma) = 1$
	assert $m_i \neq j$	Oracle $\text{SIGN}(m)$
	$\mathcal{Q} := \{m\}$	assert $\mathcal{Q} = \emptyset$
	return $\text{Sign}(pp, sk, m)$	assert $m_i \neq j$
		$\mathcal{Q} := \{m\}$
		return $\text{Sign}(pp, sk, m)$

Fig. 5.3. Sequence of games for the proof that Lamport signatures are EUF-CMA secure. Differences are highlighted.

Exercise 5.4. Prove [Proposition 5.1](#).

Exercise 5.5. Complete the proof of [Theorem 5.1](#):

1. Express $\Pr \left[\text{Game } \text{G}_1^{\mathcal{A}}(\lambda) = 1 \right]$ in terms of $\Pr \left[\text{Game } \text{EUF-CMA}_{\text{LS}[\text{H},k]}^{\mathcal{A}}(\lambda) = 1 \right]$.
2. Express $\Pr \left[\text{Game } \text{G}_2^{\mathcal{A}}(\lambda) = 1 \right]$ in terms of $\Pr \left[\text{Game } \text{G}_1^{\mathcal{A}}(\lambda) = 1 \right]$.
3. Give a p.p.t. algorithm $\mathcal{B}^{\mathcal{A}}$ that wins $\text{Game Preimage}_{\text{H}}^{\mathcal{A}}(\lambda)$ with probability $\Pr \left[\text{Game } \text{G}_2^{\mathcal{A}}(\lambda) = 1 \right]$.
4. Put it all together to prove [Theorem 5.1](#).

Exercise 5.6. Let $\text{Game } \text{G}'$ be the same as Game EUF-CMA except that the signing oracle can be called twice. Where does the proof of [Exercise 5.5](#) break?

Exercise 5.7 (Optional). Why are Lamport signatures considered post-quantum secure?

Exercise 5.8 (Optional). Consider a variant of Lamport signatures that uses an accumulator ([Definition 4.1](#)). Instead of storing $pk = (pk_{1,0}, pk_{1,1}, \dots, pk_{k,0}, pk_{k,1})$, we insert all $2k$ values $\{pk_{i,b} : i \in [1, k], b \in \{0, 1\}\}$ into an accumulator and set the public key to be the accumulator value.

1. Describe how signing and verification would work.
2. What are the trade-offs compared to standard Lamport signatures?
3. For security, what property of the accumulator is needed? Sketch how you would modify the EUF-CMA proof.

Exercise 5.9 (Optional). Describe how to build a stateful many-time signature scheme from a one-time signature scheme and an accumulator. Your construction should support signing up to n messages (where n is fixed at setup).

Exercise 5.10 (Optional). Read Sections 2.2, Theorem 1, and 3.2 of Hülsing [[Hül13](#)] about the Winternitz One-Time Signature scheme. Explain what challenges the security reduction embeds and how they are embedded.

6 Discrete Logarithm

6.1 The Discrete Logarithm Problem

Definition 6.1 (Group Generation Algorithm). A group generation algorithm GrGen is a p.p.t. algorithm that takes the security parameter 1^λ as input and outputs a group description $(\mathbb{G}, p, g)$, where $\mathbb{G}$ is a cyclic group of prime order p with $p \geq 2^{\lambda-1}$, and g is a generator of $\mathbb{G}$.

Remark 6.1. The group operation is denoted multiplicatively ($g \cdot g = g^2$).

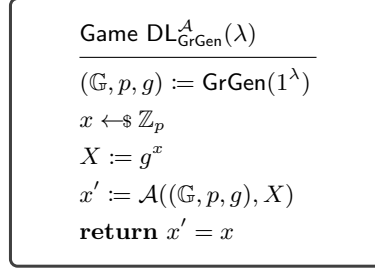


Fig. 6.1. The discrete logarithm game

Definition 6.2 (Discrete Logarithm Assumption). Let GrGen be a group generation algorithm, and let $\text{Game DL}_{\text{GrGen}}^{\mathcal{A}}(\lambda)$ be as defined in [Figure 6.1](#). The discrete logarithm (DL) problem is hard for GrGen if for any p.p.t. algorithm $\mathcal{A}$,

$$\text{Adv}_{\mathcal{A}, \text{GrGen}}^{\text{dl}}(\lambda) := \Pr \left[\text{Game DL}_{\text{GrGen}}^{\mathcal{A}}(\lambda) = 1 \right] = \text{negl}(\lambda).$$

6.2 Pedersen Commitments

Definition 6.3 (Pedersen Commitment Scheme). Let GrGen be a group generation algorithm. The Pedersen commitment scheme $\text{PedCom}[\text{GrGen}]$ is defined as follows:

- **Setup** $(1^\lambda) \rightarrow pp$: Run $(\mathbb{G}, p, g) := \text{GrGen}(1^\lambda)$, sample $h \leftarrow \mathbb{G} \setminus \{1_{\mathbb{G}}\}$, and output $pp = ((\mathbb{G}, p, g), h, \mathcal{M} := \mathbb{Z}_p, \mathcal{R} := \mathbb{Z}_p)$.
- **Commit** $(pp, m, r) \rightarrow C$: Parse $pp = ((\mathbb{G}, p, g), h, \mathcal{M}, \mathcal{R})$. For message $m \in \mathcal{M}$ and randomness $r \in \mathcal{R}$, output $C = g^m h^r$.

Remark 6.2. Pedersen commitments are *homomorphic*: for commitments $C_1 = \text{Commit}(pp, m_1, r_1)$ and $C_2 = \text{Commit}(pp, m_2, r_2)$, we have

$$C_1 \cdot C_2 = g^{m_1+m_2} h^{r_1+r_2} = \text{Commit}(pp, m_1 + m_2, r_1 + r_2).$$

Theorem 6.1 (Pedersen Commitments are Binding). Let GrGen be a group generation algorithm for which the discrete logarithm problem is hard, then the Pedersen commitment scheme $\text{PedCom}[\text{GrGen}]$ is binding. More precisely, for any p.p.t. adversary $\mathcal{A}$ against the binding property of $\text{PedCom}[\text{GrGen}]$, there exists a p.p.t. adversary $\mathcal{B}$ against the discrete logarithm problem such that

$$\text{Adv}_{\mathcal{A}, \text{PedCom}}^{\text{bind}}(\lambda) = \text{Adv}_{\mathcal{B}, \text{GrGen}}^{\text{dl}}(\lambda).$$

The proof is left as an exercise (see [Exercise 6.2](#)).

Theorem 6.2 (Pedersen Commitments are Hiding). The Pedersen commitment scheme $\text{PedCom}[\text{GrGen}]$ is perfectly hiding. More precisely, for any (possibly unbounded) adversary $\mathcal{A}$,

$$\text{Adv}_{\mathcal{A}, \text{PedCom}}^{\text{hid}}(\lambda) = 0.$$

The proof is left as an exercise (see [Exercise 6.3](#)).

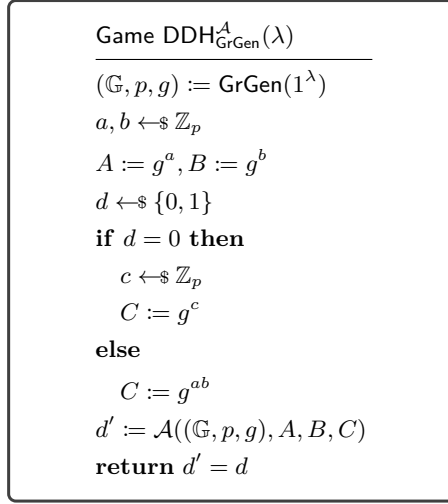


Fig. 6.2. The decisional Diffie-Hellman game

6.3 The DDH Assumption

Definition 6.4 (Decisional Diffie-Hellman Assumption). Let GrGen be a group generation algorithm, and let $\text{Game DDH}_{\text{GrGen}}^A(\lambda)$ be as defined in Figure 6.2. The decisional Diffie-Hellman (DDH) problem is hard for GrGen if for any p.p.t. algorithm $\mathcal{A}$,

$$\text{Adv}_{\mathcal{A}, \text{GrGen}}^{\text{ddh}}(\lambda) := \left| \Pr \left[\text{Game DDH}_{\text{GrGen}}^A(\lambda) = 1 \right] - \frac{1}{2} \right| = \text{negl}(\lambda).$$

6.4 ElGamal Commitments

Definition 6.5 (ElGamal Commitment Scheme). Let GrGen be a group generation algorithm. The ElGamal commitment scheme $\text{ElGamalCom}[\text{GrGen}]$ is defined as follows:

- **Setup**(1^λ) $\rightarrow$ pp : Run $(\mathbb{G}, p, g) := \text{GrGen}(1^\lambda)$, sample $h \leftarrow \mathbb{G} \setminus \{1_{\mathbb{G}}\}$, and output $pp = ((\mathbb{G}, p, g), h, \mathcal{M} := \mathbb{Z}_p, \mathcal{R} := \mathbb{Z}_p)$.
- **Commit**(pp, m, r) $\rightarrow$ C : Parse $pp = ((\mathbb{G}, p, g), h, \mathcal{M}, \mathcal{R})$. For message $m \in \mathcal{M}$ and randomness $r \in \mathcal{R}$, output $C = (g^r, h^{m+r})$.

Theorem 6.3 (ElGamal Commitments are Hiding). Let GrGen be a group generation algorithm for which the DDH problem is hard, then the ElGamal commitment scheme $\text{ElGamalCom}[\text{GrGen}]$ is hiding. More precisely, for any p.p.t. adversary $\mathcal{A}$ against the hiding property of $\text{ElGamalCom}[\text{GrGen}]$, there exists a p.p.t. adversary $\mathcal{B}$ against DDH such that

$$\text{Adv}_{\mathcal{A}, \text{ElGamalCom}}^{\text{hide}}(\lambda) = 2 \cdot \text{Adv}_{\mathcal{B}, \text{GrGen}}^{\text{ddh}}(\lambda).$$

The proof is left as an exercise (see Exercise 6.4).

6.5 The OMDL and AOMDL Problems

Definition 6.6 (One-More Discrete Logarithm Assumption). Let GrGen be a group generation algorithm, and let $\text{Game OMDL}_{\text{GrGen}}^A(\lambda)$ be as defined in Figure 6.3. The one-more discrete logarithm (OMDL) problem is hard for GrGen if for any p.p.t. algorithm $\mathcal{A}$,

$$\text{Adv}_{\mathcal{A}, \text{GrGen}}^{\text{omdl}}(\lambda) := \Pr \left[\text{Game OMDL}_{\text{GrGen}}^A(\lambda) = 1 \right] = \text{negl}(\lambda).$$

Game $\text{OMDL}_{\text{GrGen}}^A(\lambda)$	Oracle $\text{CHAL}()$	Game $\text{AOMDL}_{\text{GrGen}}^A(\lambda)$
$(\mathbb{G}, p, g) := \text{GrGen}(1^\lambda)$	$\ell := \ell + 1$	$(\mathbb{G}, p, g) := \text{GrGen}(1^\lambda)$
$\ell := 0; q := 0$	$x_\ell \leftarrow \mathbb{Z}_p$	$\ell := 0; q := 0$
$\vec{y} := \mathcal{A}^{\text{CHAL}, \text{DL}}((\mathbb{G}, p, g))$	$X_\ell := g^{x_\ell}$	$\vec{y} := \mathcal{A}^{\text{CHAL}, \text{ADL}}((\mathbb{G}, p, g))$
$\vec{x} := (x_1, \dots, x_\ell)$	return X_ℓ	$\vec{x} := (x_1, \dots, x_\ell)$
return $(\vec{y} = \vec{x} \wedge q < \ell)$		return $(\vec{y} = \vec{x} \wedge q < \ell)$
Oracle $\text{DL}(X)$		Oracle $\text{ADL}((\alpha, \beta_1, \dots, \beta_\ell))$
$q := q + 1$		$q := q + 1$
return $\log_g(X)$		return $\alpha + \sum_{i=1}^\ell \beta_i x_i$
		$// = \log_g(g^\alpha \prod_{i=1}^\ell X_i^{\beta_i})$

Fig. 6.3. The one-more discrete logarithm (OMDL) and algebraic one-more discrete logarithm (AOMDL) games. The difference is highlighted.

Definition 6.7 (Algebraic One-More Discrete Logarithm Assumption). Let GrGen be a group generation algorithm, and let $\text{Game AOMDL}_{\text{GrGen}}^A(\lambda)$ be as defined in Figure 6.3. The algebraic one-more discrete logarithm (AOMDL) problem is hard for GrGen if for any p.p.t. algorithm $\mathcal{A}$,

$$\text{Adv}_{\mathcal{A}, \text{GrGen}}^{\text{aomdl}}(\lambda) := \Pr \left[\text{Game AOMDL}_{\text{GrGen}}^A(\lambda) = 1 \right] = \text{negl}(\lambda).$$

Remark 6.3. When proving the security of a scheme using the AOMDL assumption, the ADL oracle is (typically) used by the reduction and not by the adversary against the scheme. Therefore, the security proof is not restricted to adversaries that know the algebraic representations of all group elements that they output.

Remark 6.4 (Falsifiable Assumptions). In order to be able to evaluate whether an assumption is true or not, it must be falsifiable, i.e., there must be a (constructive) way to demonstrate that it is false, if this is the case. More precisely, a cryptographic assumption is *falsifiable* if there exists a p.p.t. challenger algorithm that interacts with an adversary and decides whether the adversary breaks the assumption.

6.6 Exercises

Exercise 6.1. In the framework of asymptotic security, why is the DL problem on secp256k1 not hard? What does GrGen do?

Exercise 6.2. Argue that Pedersen commitments are binding under the DL assumption (Theorem 6.1).

Exercise 6.3. Prove that Pedersen commitments are perfectly hiding (Theorem 6.2).

Exercise 6.4. Prove that ElGamal commitments are hiding under the DDH assumption (Theorem 6.3).

Exercise 6.5. Argue that if OMDL is hard for a group generation algorithm then DL is hard.

Exercise 6.6. Consider an algorithm $\mathcal{A}$ playing the AOMDL game. It queries CHAL twice to obtain X_1 and X_2 , then samples $\alpha, \beta_1, \beta_2 \leftarrow \mathbb{Z}_p$.

1. How should $\mathcal{A}$ call the ADL oracle to obtain $\log_g(P)$ where $P = g^\alpha X_1^{\beta_1} X_2^{\beta_2}$?
2. Assuming $\mathcal{A}$ makes no more CHAL queries, how many more ADL queries can it make?
3. What must $\mathcal{A}$ return to win the game?

Exercise 6.7. Is the OMDL assumption falsifiable? Is the AOMDL assumption falsifiable? Do we prefer the security of a scheme to be based on OMDL or AOMDL?

Exercise 6.8 (Optional). Why is the DL problem not hard in general?

Exercise 6.9 (Optional). Read about the Generic Group Model (GGM) in the introduction of [Sho97] and [Mau05], and argue why the DLP is hard for classical computational models.

Exercise 6.10 (Optional). How does Silent Payments [BIP352] rely on the DDH assumption?

Exercise 6.11 (Optional). Study the concept of falsifiability of cryptographic assumptions by reading Naor [Nao03]. Analyze the following assumptions and explain the extent to which each one is falsifiable:

- Factoring
- The RSA assumption
- The Decisional Diffie-Hellman assumption
- The Knowledge-of-Exponent assumption

7 Random Oracle Model

7.1 Random Oracles

A *random oracle* (RO) is an oracle that can be provided to an algorithm that takes inputs in $\{0, 1\}^*$ and returns elements from some set S . The two key properties of a random oracle are: (1) it returns uniformly random and independent values from S for distinct queries, and (2) it responds consistently, meaning identical queries always yield identical responses.

Formally, a random oracle can be described as follows. Consider the random oracle H in Figure 7.1. The oracle maintains a table T to ensure consistency, where all entries are initially $\perp$ (undefined). When queried with input x , it checks whether $T[x] = \perp$. If so, it samples a uniformly random element from S and stores it as $T[x]$. The oracle returns $T[x]$.

Game $\text{ROExample}^A(\lambda)$	Oracle $H(x)$
$T := \emptyset$	if $T[x] = \perp$ then
$r \leftarrow_{\$} \{0, 1\}^\lambda$	$T[x] \leftarrow_{\$} S$
$h := H(r)$	return $T[x]$
return $\mathcal{A}^H(h)$	

Fig. 7.1. Random oracle example game

Before examining the consequences of random oracles, we present a simple lemma that will be useful for analyzing games that differ only when certain events occur.

Lemma 7.1 (Difference Lemma). *Let A , B and E be events in a probability space. If $\Pr[A \wedge \neg E] = \Pr[B \wedge \neg E]$, then*

$$|\Pr[A] - \Pr[B]| \leq \Pr[E].$$

Proof.

$$\begin{aligned} |\Pr[A] - \Pr[B]| &= |\Pr[A \wedge E] + \Pr[A \wedge \neg E] - (\Pr[B \wedge E] + \Pr[B \wedge \neg E])| \\ &= |\Pr[A \wedge E] - \Pr[B \wedge E]| \end{aligned}$$

Since $\Pr[A \wedge E] \leq \Pr[E]$ and $\Pr[B \wedge E] \leq \Pr[E]$, both terms are in $[0, \Pr[E]]$, so their difference is at most $\Pr[E]$. $\square$

Lemma 7.2. *No adversary against ROExample can distinguish between being given $h = H(r)$ for a random r and being given a uniformly random value from S , except with probability negligible in the number of oracle queries. More precisely, let $\text{Game ROExample}^{\mathcal{A}}(\lambda)$ be as defined in Figure 7.1. For any adversary $\mathcal{A}$ making q queries to the random oracle H , we have*

$$\left| \Pr[\text{ROExample}^{\mathcal{A}}(\lambda) = 1] - \Pr[\text{ROExample}_1^{\mathcal{A}}(\lambda) = 1] \right| \leq \frac{q}{2^\lambda}$$

where $\text{ROExample}_1^{\mathcal{A}}(\lambda)$ is defined in Figure 7.2.

Game $\text{ROExample}_1^{\mathcal{A}}(\lambda)$	Oracle $H(x)$
$T := \emptyset$	assert $x \neq r$
$r \leftarrow_{\$} \{0, 1\}^\lambda$	if $T[x] = \perp$ then
	$T[x] \leftarrow_{\$} S$
$h' \leftarrow_{\$} S$	return $T[x]$
return $\mathcal{A}^H(h')$	

Fig. 7.2. Game hop for random oracle indistinguishability

Proof. By inspection of the code, we observe that $\text{ROExample}_1^{\mathcal{A}}(\lambda)$ is identical to $\text{ROExample}^{\mathcal{A}}(\lambda)$ except that:

1. The input to $\mathcal{A}$ is changed from $h = H(r)$ to a uniformly random $h' \leftarrow_{\$} S$.
2. The oracle H asserts that no query equals r .

Since h' is uniformly random in S and has the same distribution as $H(r)$ (when r is not queried), the games are identical unless the adversary queries r to the oracle.

We apply the Difference Lemma (Lemma 7.1). Let E denote the event that $\mathcal{A}$ queries r to the oracle.

When $\neg E$ occurs (i.e., r is not queried), the two games behave identically, so $\Pr[\text{ROExample}^{\mathcal{A}}(\lambda) = 1 \wedge \neg E] = \Pr[\text{ROExample}_1^{\mathcal{A}}(\lambda) = 1 \wedge \neg E]$.

By Lemma 7.1:

$$\begin{aligned} \left| \Pr[\text{ROExample}^{\mathcal{A}}(\lambda) = 1] - \Pr[\text{ROExample}_1^{\mathcal{A}}(\lambda) = 1] \right| &\leq \Pr[E] \\ &= \Pr[\exists i \in [q] : x_i = r] \\ &\leq \sum_{i=1}^q \Pr[x_i = r] \quad (\text{union bound}) \\ &= \sum_{i=1}^q \frac{1}{2^\lambda} = \frac{q}{2^\lambda} \end{aligned}$$

where $x_1, \dots, x_q$ are the queries made by $\mathcal{A}$ to H . □

7.2 ROM

For a large class of cryptographic schemes, the properties of hash functions seen so far are insufficient to prove security. Instead, we prove security of a modified scheme where hash function evaluations are replaced with random oracle calls. This is the *random oracle model* (ROM) [BR93]. Random oracles cannot be instantiated in the real world because their output must be distributed uniformly at random unless explicitly queried. On the other hand, the output of a hash function is deterministic

once the key is known. Moreover, there's no way to formally define an “explicit query” for public hash functions.

There exist contrived schemes provably secure in the ROM that become insecure when instantiated with any concrete hash function (see optional exercise). Despite these theoretical concerns, the ROM has proven very useful in practice for analyzing real-world cryptographic protocols [KM15].

7.3 Hash Commitments

Definition 7.1 (Hash Commitment Scheme). Let H be a hash function. The hash commitment scheme $\text{HashCom}[H]$ is defined as follows:

- $\text{Setup}(1^\lambda) \rightarrow pp$: Output $pp = (\mathcal{M} := \{0, 1\}^*, \mathcal{R} := \{0, 1\}^\lambda)$.
- $\text{Commit}(pp, m, r) \rightarrow C$: Output $C = H.\text{Eval}(m||r)$.

Lemma 7.3. The hash commitment scheme $\text{HashCom}[H]$ is hiding in the random oracle model for H . More precisely, for any algorithm $\mathcal{A}$ making at most q queries to H ,

$$\text{Adv}_{\mathcal{A}, \text{HashCom}}^{\text{hide}}(\lambda) \leq \frac{2q}{2^\lambda}.$$

The proof is left as an exercise (see [Exercise 7.2](#)).

7.4 Taproot Commitments

Definition 7.2 (Taproot Commitment Scheme [BIP341]). Let GrGen be a group generation algorithm and H be a hash function with output in $\mathbb{Z}_p$ where p is the group order. The Taproot commitment scheme $\text{TapCom}[\text{GrGen}, H]$ is defined as follows:

- $\text{Setup}(1^\lambda) \rightarrow pp$: Run $(\mathbb{G}, p, g) := \text{GrGen}(1^\lambda)$ and $\kappa := H.\text{Gen}(1^\lambda)$. Output $pp = ((\mathbb{G}, p, g), \kappa, \mathcal{M} := \{0, 1\}^*, \mathcal{R} := \mathbb{G})$.
- $\text{Commit}(pp, m, R) \rightarrow C$: Parse pp to obtain g and κ . Output $C = R \cdot g^{H.\text{Eval}(\kappa, (R, m))}$.

Lemma 7.4. Let GrGen be a group generation algorithm and H be a hash function with output in $\mathbb{Z}_p$ where p is the group order. The Taproot commitment scheme $\text{TapCom}[\text{GrGen}, H]$ is binding in the random oracle model for H . More precisely, for any algorithm $\mathcal{A}$ making at most q queries to H ,

$$\text{Adv}_{\mathcal{A}, \text{TapCom}}^{\text{bind}}(\lambda) \leq \frac{(q+2)^2}{2^\lambda}.$$

The proof is left as an exercise (see [Exercise 7.3](#)).

7.5 Exercises

Exercise 7.1. Consider a modified hash commitment scheme $\text{HashCom}_y[H]$ defined as follows:

- $\text{Setup}(1^\lambda) \rightarrow pp$: Same as $\text{HashCom}[H]$.
- $\text{Commit}(pp, m, r) \rightarrow C$:

$$C = \begin{cases} H.\text{Eval}(m) & \text{if } H.\text{Eval}(0) = y \\ H.\text{Eval}(m||r) & \text{otherwise} \end{cases}$$

1. Using Lemma 7.3, argue that for all $y \in \{0, 1\}^\lambda$, $\text{HashCom}_y[H]$ is hiding in the ROM.
2. Show that there exists y such that HashCom_y instantiated with SHA256 is not hiding.

Exercise 7.2. Prove Lemma 7.3 using a similar technique as in Lemma 7.2.

Exercise 7.3. Prove Lemma 7.4.

Exercise 7.4 (Optional).

1. Let H be a hash function with output in $\{0,1\}^\lambda$ and let κ be chosen uniformly at random from a set of size 2^λ . Define $A = H(\text{Eval}(\kappa, 0)) \parallel H(\text{Eval}(\kappa, 1))$. What is the size of the set over which A is uniformly distributed?
2. Let H be a random oracle with output in $\{0,1\}^\lambda$. Define $B = H(0) \parallel H(1)$. What is the size of the set over which B is uniformly distributed?

Exercise 7.5 (Optional). Let H be a random oracle with output in $\{0,1\}^\lambda$. Define the function $G : (\{0,1\}^*)^{\lambda+1} \rightarrow \{0,1\}^\lambda$ as:

$$G(x_0, x_1, \dots, x_\lambda) = H(x_0) \oplus H(x_1) \oplus \dots \oplus H(x_\lambda)$$

where $\oplus$ denotes bitwise XOR. Is G preimage resistant? That is, given $y \in \{0,1\}^\lambda$, is it hard for a p.p.t. adversary with oracle access to H to find $(x_0, \dots, x_\lambda)$ such that $G(x_0, \dots, x_\lambda) = y$?

Exercise 7.6 (Optional). Why is the ROM not considered adequate to model quantum computation?

Exercise 7.7 (Optional). Study the diagonalization argument in Section 4 of Canetti, Goldreich, and Halevi [CGH98]. Explain how this argument demonstrates the existence of a contrived signature scheme that is provably secure in the random oracle model but becomes insecure when the random oracle is instantiated with any concrete hash function (not just a specific fixed hash function as considered in Exercise 7.1).

8 Recap Quiz

Exercise 8.1. Let's say you have a game **Game G** that you want to prove is hard, and you have the option to upper bound $\Pr[G_A^\lambda = 1]$ by $\text{Adv}_{B, \text{GrGen}}^{\text{aomdl}}(\lambda)$, $\text{Adv}_{C, \text{GrGen}}^{\text{omdl}}(\lambda)$, $\text{Adv}_{D, \text{GrGen}}^{\text{omdl}}(\lambda)$ in the ROM, or $\text{Adv}_{E, \text{GrGen}}^{\text{dl}}(\lambda)$. What is your order of preference?

1. $\text{Adv}_{E, \text{GrGen}}^{\text{dl}}(\lambda)$, $\text{Adv}_{B, \text{GrGen}}^{\text{aomdl}}(\lambda)$, $\text{Adv}_{C, \text{GrGen}}^{\text{omdl}}(\lambda)$, $\text{Adv}_{D, \text{GrGen}}^{\text{omdl}}(\lambda)$ in the ROM
2. $\text{Adv}_{E, \text{GrGen}}^{\text{dl}}(\lambda)$, $\text{Adv}_{C, \text{GrGen}}^{\text{omdl}}(\lambda)$, $\text{Adv}_{B, \text{GrGen}}^{\text{aomdl}}(\lambda)$, $\text{Adv}_{D, \text{GrGen}}^{\text{omdl}}(\lambda)$ in the ROM
3. $\text{Adv}_{D, \text{GrGen}}^{\text{omdl}}(\lambda)$ in the ROM, $\text{Adv}_{E, \text{GrGen}}^{\text{dl}}(\lambda)$, $\text{Adv}_{B, \text{GrGen}}^{\text{aomdl}}(\lambda)$, $\text{Adv}_{C, \text{GrGen}}^{\text{omdl}}(\lambda)$
4. $\text{Adv}_{B, \text{GrGen}}^{\text{aomdl}}(\lambda)$, $\text{Adv}_{E, \text{GrGen}}^{\text{dl}}(\lambda)$, $\text{Adv}_{C, \text{GrGen}}^{\text{omdl}}(\lambda)$, $\text{Adv}_{D, \text{GrGen}}^{\text{omdl}}(\lambda)$ in the ROM

Exercise 8.2. Consider the following game hop from **Game $\text{Guess}^A(\lambda)$** to **Game $\text{Guess}_2^A(\lambda)$** :

Game $\text{Guess}_2^A(\lambda)$

$n := 2^\lambda$

$x \leftarrow \mathbb{Z}_n$

assert $x \neq 0$

$x' := \mathcal{A}()$

return $x = x'$

Fig. 8.1. Without the **highlighted** change, this is the guessing game from Figure 1.1. With the change, this is **Game $\text{Guess}_2^A(\lambda)$** .

Which of the following is correct?

1. $|\text{Adv}_{\mathcal{A}}^{\text{guess}}(\lambda) - \Pr[\text{Guess}_2^A(\lambda) = 1]| \leq \Pr[x = 0]$
2. $|\text{Adv}_{\mathcal{A}}^{\text{guess}}(\lambda) - \Pr[\text{Guess}_2^A(\lambda) = 1]| = \Pr[x = 0]$
3. $|\text{Adv}_{\mathcal{A}}^{\text{guess}}(\lambda) - \Pr[\text{Guess}_2^A(\lambda) = 1]| \geq \Pr[x = 0]$

Exercise 8.3. Using the result from the previous exercise, what can we conclude about $\Pr[\text{Guess}_2^A(\lambda) = 1]$?

1. $\Pr[\text{Guess}_2^{\mathcal{A}}(\lambda) = 1] \geq \text{Adv}_{\mathcal{A}}^{\text{guess}}(\lambda) - 2^{-\lambda}$
2. $\Pr[\text{Guess}_2^{\mathcal{A}}(\lambda) = 1] \geq \text{Adv}_{\mathcal{A}}^{\text{guess}}(\lambda) + 2^{-\lambda}$
3. $\Pr[\text{Guess}_2^{\mathcal{A}}(\lambda) = 1] < \text{Adv}_{\mathcal{A}}^{\text{guess}}(\lambda) - 2^{-\lambda}$

Exercise 8.4. Let $\mathcal{A}$ be an algorithm with access to a random oracle $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$. The algorithm queries the random oracle with $x \in \{0, 1\}^\lambda$. What is $\Pr[H(x) = x]$?

1. 0
2. $2^{-\lambda}$
3. 1
4. $2^{-2\lambda}$

Exercise 8.5. Let p be a prime and let $\mathcal{A}$ be an algorithm with access to a random oracle $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p$. Before returning, the adversary queries the random oracle with distinct $x_1, x_2 \in \mathbb{Z}_p$ (in that order) and receives $y_1 = H(x_1)$, $y_2 = H(x_2)$. What is $\Pr[y_2 = x_1 y_1 x_2^{-1}]$?

1. 0
2. $1/p$
3. $1/(p-1)$
4. 1

Exercise 8.6. Let $(\mathbb{G}, p, g)$ be a group description. Let $\mathcal{A}$ be an algorithm with access to a random oracle $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ where p is prime. The adversary makes two distinct queries (R_0, m_0) and (R_1, m_1) where $R_i \in \mathbb{G}$ and $m_i \in \{0, 1\}^*$, receiving $y_0 = H(R_0, m_0)$ and $y_1 = H(R_1, m_1)$. What is $\Pr[R_0 \cdot g^{y_0} \cdot R_1^{-1} = g^{y_1}]$?

1. 0
2. $1/p$
3. $1/(p-1)$
4. 1

Exercise 8.7. TODO: Insert question about reading random oracle queries (table T), which is not possible with hash functions.

Exercise 8.8. Which of the following represents a gap between security proofs and real-world security?

1. In practice, λ is a fixed value (e.g., 128 or 256)
2. Concrete hash functions like SHA256 are not hash functions
3. Unfalsifiable assumptions cannot be validated experimentally
4. Security models may not capture all attack vectors
5. Random oracles do not exist
6. Security proofs don't model users writing keys on post-it notes
7. All of the above

9 Programmable ROM and Forking Lemma

9.1 Programmable Random Oracles

In the random oracle model, we can program the oracle to return specific values for certain inputs, as long as we maintain consistency and the correct distribution. This technique is particularly useful in security reductions.

For example, consider the hash commitment scheme $\text{HashCom}[H]$ where commitments are of the form $H(m||r)$. A reduction can program the oracle such that certain algebraic relations hold between committed messages, while maintaining the adversary's view.

The following proposition demonstrates that careful programming preserves the adversary's advantage. Notably, Game_1 forces the XOR of committed messages to be equal to a message m^* that is determined before even running the adversary. The game first draws m^* , runs the adversary

with a random hash h and obtains a hash h' from the adversary. If the adversary created h' as $\text{Commit}(m', r')$, then when the adversary is invoked a second time we have

$$\begin{aligned}\text{Commit}(m, r) &= h \\ \text{Commit}(m', r') &= h' \\ m \oplus m' &= m^*.\end{aligned}$$

Note that the challenger's commitment h was given to the adversary before the adversary returned commitment h' and the adversary has never revealed the opening (m', r') to the challenger.

A similar technique is used in MuSig multi-signature scheme [Max+19] to simulate signing queries (i.e., to answer signing queries without knowledge of the secret key, as explained in the next section).

$\text{Game}_0^{\mathcal{A}}(\lambda)$	Oracle $H(x)$	$\text{Game}_1^{\mathcal{A}}(\lambda)$
$pp := \text{HashCom.Setup}(1^\lambda)$	if $T[x] = \perp$ then	$pp := \text{HashCom.Setup}(1^\lambda)$
$T := \emptyset$	$T[x] \leftarrow_{\$} \{0, 1\}^\lambda$	$T := \emptyset$
$m \leftarrow_{\$} \{0, 1\}$	return $T[x]$	$m^* \leftarrow_{\$} \{0, 1\}$
$r \leftarrow_{\$} \{0, 1\}^\lambda$		$h \leftarrow_{\$} \{0, 1\}^\lambda$
$h := H(m \ r)$		$(\text{state}, h') := \mathcal{A}^H(pp, h)$
$(\text{state}, h') := \mathcal{A}^H(pp, h)$		if $\exists(m', r') : T[m' \ r'] = h'$ then
return $\mathcal{A}^H(\text{state}, m, r)$		Choose any such (m', r')
		$m := m^* \oplus m'$
		else
		$m \leftarrow_{\$} \{0, 1\}$
		$r \leftarrow_{\$} \{0, 1\}^\lambda$
		assert $T[m \ r] = \perp$
		$T[m \ r] := h$
		return $\mathcal{A}^H(\text{state}, m, r)$

Fig. 9.1. Games Game_0 and Game_1

Proposition 9.1. *Let $\text{Game}_0^{\mathcal{A}}(\lambda)$ and $\text{Game}_1^{\mathcal{A}}(\lambda)$ be as defined in Figure 9.1. For any adversary $\mathcal{A}$ making at most q queries to the oracle H , we have*

$$\left| \Pr[\text{Game}_0^{\mathcal{A}}(\lambda) = 1] - \Pr[\text{Game}_1^{\mathcal{A}}(\lambda) = 1] \right| \leq \frac{q}{2^\lambda}$$

The proof is left as an exercise (see Exercise 9.3).

9.2 The Forking Lemma

The forking lemma is a fundamental tool for analyzing algorithms that interact with random oracles. It considers an adversary $\mathcal{A}$ with oracle access to H , which is executed twice on different but related instances of the oracle:

- In the first execution, all random oracle responses are sampled uniformly as usual.
- In the second execution, the responses are identical to those from the first execution up to a specific query called the *forking point*, whose response is *refreshed* (resampled with fresh randomness).

As a result, the two executions of $\mathcal{A}$ proceed identically up to the forking point but diverge afterwards. The forking lemma quantifies how often an adversary that succeeds in the first execution also succeeds in the second execution, where the oracle gives a different response at the forking point.

Game $\text{Single}_{\text{InpGen}}^{\mathcal{A}}(\lambda)$	Game $\text{Fork}_{\text{InpGen}}^{\mathcal{A}}(\lambda)$
$inp := \text{InpGen}(1^\lambda)$	$inp := \text{InpGen}(1^\lambda)$
$\rho \leftarrow \$ \mathcal{R}$	$\rho \leftarrow \$ \mathcal{R}$
$T := \emptyset$	$T := \emptyset$
$\alpha := \mathcal{A}^H(inp; \rho)$	$\alpha := \mathcal{A}^H(inp; \rho)$
assert $\alpha \neq \perp$	assert $\alpha \neq \perp$
return 1	$(z, aux) := \alpha$
Oracle $H(x)$	$T' := T \quad // \text{ Copy table}$
if $T[x] = \perp$ then	$T'[z] \leftarrow \$ S$
$T[x] \leftarrow \$ S$	$\alpha' := \mathcal{A}^H(inp; \rho)$
return $T[x]$	assert $\alpha' \neq \perp$
	$(z', aux') := \alpha'$
	assert $z = z' \wedge T[z] \neq T'[z]$
	return (z, aux, z', aux')
	Oracle $H'(x)$
	if $T'[x] = \perp$ then
	$T'[x] \leftarrow \$ S$
	return $T'[x]$

Fig. 9.2. The Single and Fork games

Lemma 9.1 (Local Forking Lemma [BDL19]). *Let $q \geq 1$ be an integer and InpGen be a randomized algorithm that outputs some input inp . Let $\mathcal{A}$ be a randomized algorithm that takes input inp and randomness $\rho \in \mathcal{R}$, has oracle access to $H : \{0, 1\}^* \rightarrow S$ where $|S| \geq 2^\lambda$, makes at most q queries to H , and returns either $\perp$ or a pair (z, aux) where $z \in \{0, 1\}^*$ and aux is auxiliary output.*

For games $\text{Single}_{\text{InpGen}}^{\mathcal{A}}(\lambda)$ and $\text{Fork}_{\text{InpGen}}^{\mathcal{A}}(\lambda)$ defined in Figure 9.2, define:

$$\text{Adv}_{\mathcal{A}, \text{InpGen}}^{\text{single}}(\lambda) := \Pr[\text{Single}_{\text{InpGen}}^{\mathcal{A}}(\lambda) = 1]$$

$$\text{Adv}_{\mathcal{A}, \text{InpGen}}^{\text{fork}}(\lambda) := \Pr[\text{Fork}_{\text{InpGen}}^{\mathcal{A}}(\lambda) \neq \perp]$$

Then:

$$\text{Adv}_{\mathcal{A}, \text{InpGen}}^{\text{fork}}(\lambda) \geq \text{Adv}_{\mathcal{A}, \text{InpGen}}^{\text{single}}(\lambda) \left(\frac{\text{Adv}_{\mathcal{A}, \text{InpGen}}^{\text{single}}(\lambda)}{q} - \frac{1}{2^\lambda} \right)$$

9.3 Exercises

Exercise 9.1. Describe the similarities and differences between games Game_0 and Game_1 in Figure 9.1.

Exercise 9.2. Consider Game_F which is exactly like Game_1 except that $m^* := 0$ (instead of sampling uniformly). Give a p.p.t. adversary $\mathcal{A}$ that makes one random oracle query and achieves

$$\left| \Pr[\text{Game}_0^{\mathcal{A}}(\lambda) = 1] - \Pr[\text{Game}_F^{\mathcal{A}}(\lambda) = 1] \right| = \frac{1}{2}.$$

Exercise 9.3. Sketch a proof of Proposition 9.1. **Hints:**

1. Let E be the event that the adversary queries $m||r$ to the oracle during its first invocation (i.e., the event that triggers the assertion). Argue that $\Pr[\text{Game}_0^{\mathcal{A}}(\lambda) = 1 \wedge \neg E] = \Pr[\text{Game}_1^{\mathcal{A}}(\lambda) = 1 \wedge \neg E]$.
2. Bound the probability of E .

Exercise 9.4. Consider an algorithm $\mathcal{A}$ that never queries H . What is $\text{Adv}_{\mathcal{A}, \text{InpGen}}^{\text{fork}}(\lambda)$?

Exercise 9.5. Consider an algorithm $\mathcal{A}$ that makes exactly one query $z := H(\rho)$ and outputs $(z, \perp)$. What is $\text{Adv}_{\mathcal{A}, \text{InpGen}}^{\text{fork}}(\lambda)$?

Exercise 9.6. Let InpGen output $\text{inp} = (\mathbb{G}, p, g)$ where g is a generator of a cyclic group $\mathbb{G}$ of prime order $p \geq 2^{\lambda-1}$. Let $\mathcal{A}$ be an algorithm that makes at most q queries to a random oracle $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p$ and always succeeds in outputting (z, s) such that $s = z + H(z) \cdot x \bmod p$ for some fixed $x \in \mathbb{Z}_p$. Using the forking lemma, give a p.p.t. algorithm $\mathcal{B}$ that outputs x and bound its success probability.

Exercise 9.7. Consider an algorithm $\mathcal{A}$ that queries $h_0 := H(0)$ and outputs $(0, \perp)$ if $h_0 \bmod 2 = 0$, otherwise outputs $(h_0, \perp)$. What is $\text{Adv}_{\mathcal{A}, \text{InpGen}}^{\text{fork}}(\lambda)$?

Exercise 9.8 (Optional). Find an algorithm $\mathcal{A}$ making exactly q queries for which $\text{Adv}_{\mathcal{A}, \text{InpGen}}^{\text{fork}}(\lambda)$ is as close to $\text{Adv}_{\mathcal{A}, \text{InpGen}}^{\text{single}}(\lambda) \left(\frac{\text{Adv}_{\mathcal{A}, \text{InpGen}}^{\text{single}}(\lambda)}{q} - \frac{1}{2^\lambda} \right)$ as possible.

Exercise 9.9 (Optional). Compare the Generalized Forking Lemma of Bellare and Neven [BN06] with the Local Forking Lemma. What are the key differences in their formulations and applications?

10 Signatures

10.1 Syntax & Correctness

Definition 10.1 (Signature Scheme). A signature scheme Sig is a tuple of polynomial-time algorithms $(\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$ where:

- $\text{Setup}(1^\lambda) \rightarrow pp$ is a probabilistic algorithm that takes the security parameter as input and outputs public parameters pp including the message space $\mathcal{M}$ and the signature space $\mathcal{S}$.
- $\text{KeyGen}(pp) \rightarrow (sk, pk)$ is a probabilistic algorithm that takes public parameters pp as input and outputs a secret key sk and a public key pk .
- $\text{Sign}(pp, sk, m) \rightarrow \sigma$ is a probabilistic algorithm that takes public parameters pp , a secret key sk , and a message $m \in \mathcal{M}$ as input and outputs a signature $\sigma \in \mathcal{S}$.
- $\text{Verify}(pp, pk, m, \sigma) \rightarrow \{0, 1\}$ is a deterministic algorithm that takes public parameters pp , a public key pk , a message $m \in \mathcal{M}$, and a signature $\sigma \in \mathcal{S}$ as input and outputs 1 (accept) if the signature is valid, and outputs 0 (reject) otherwise.

Definition 10.2 (Correctness). A signature scheme $\text{Sig} = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$ is correct if for all $\lambda \in \mathbb{N}$, all $pp \in \text{Setup}(1^\lambda)$, all $(sk, pk) \in \text{KeyGen}(pp)$, and all messages $m \in \mathcal{M}$:

$$\Pr[\text{Verify}(pp, pk, m, \text{Sign}(pp, sk, m)) = 1] = 1$$

where the probability is taken over the randomness of Sign .

In other words, a signature scheme is correct if honestly generated signatures always verify correctly.

10.2 Security

The standard security notion for digital signatures is existential unforgeability under chosen message attack (EUF-CMA). Unlike one-time signatures, the adversary can request signatures on multiple messages.

Definition 10.3 (EUF-CMA Security). A signature scheme Sig is existentially unforgeable under chosen message attack (EUF-CMA) if for all p.p.t. adversaries $\mathcal{A}$:

$$\text{Adv}_{\mathcal{A}, \text{Sig}}^{\text{euf-cma}}(\lambda) := \Pr \left[\text{Game EUF-CMA}_{\text{Sig}}^{\mathcal{A}}(\lambda) = 1 \right] = \text{negl}(\lambda)$$

where $\text{Game EUF-CMA}_{\text{Sig}}^{\mathcal{A}}(\lambda)$ is defined in Figure 10.1.

Game $\text{EUF-CMA}_{\text{Sig}}^{\mathcal{A}}(\lambda)$	Oracle $\text{SIGN}(m)$
$pp := \text{Setup}(1^\lambda)$	$\sigma := \text{Sign}(pp, sk, m)$
$(sk, pk) := \text{KeyGen}(pp)$	$\mathcal{Q} := \mathcal{Q} \cup \{m\}$
$\mathcal{Q} := \emptyset$	return σ
$(m^*, \sigma^*) := \mathcal{A}^{\text{SIGN}}(pp, pk)$	
return $m^* \notin \mathcal{Q} \wedge$ $\text{Verify}(pp, pk, m^*, \sigma^*) = 1$	

Fig. 10.1. The EUF-CMA security game for digital signatures

10.3 Schnorr Signatures

Definition 10.4 (Schnorr Signature Scheme). Let GrGen be a group generation algorithm and H be a hash function with output space $\mathbb{Z}_p$. The Schnorr signature scheme $\text{SchnorrSig}[\text{GrGen}, H]$ is defined as follows:

- $\text{Setup}(1^\lambda) \rightarrow pp$: Run $(\mathbb{G}, p, g) := \text{GrGen}(1^\lambda)$ and $\kappa := H.\text{Gen}(1^\lambda)$. The message space is $\mathcal{M} = \{0, 1\}^*$ and the signature space is $\mathcal{S} = \mathbb{G} \times \mathbb{Z}_p$. Output $pp = ((\mathbb{G}, p, g), \kappa, \mathcal{M}, \mathcal{S})$.
- $\text{KeyGen}(pp) \rightarrow (sk, pk)$: Sample $x \leftarrow \mathbb{Z}_p$. Compute $X = g^x$. Output $sk = x$ and $pk = X$.
- $\text{Sign}(pp, sk, m) \rightarrow \sigma$: Parse $pp = ((\mathbb{G}, p, g), \kappa, \mathcal{M}, \mathcal{S})$ and $sk = x$. Sample $r \leftarrow \mathbb{Z}_p$. Compute:

$$\begin{aligned} R &= g^r \\ c &= H.\text{Eval}(\kappa, (R, m)) \\ s &= r + c \cdot x \bmod p \end{aligned}$$

Output $\sigma = (R, s)$.

- $\text{Verify}(pp, pk, m, \sigma) \rightarrow \{0, 1\}$: Parse $pp = ((\mathbb{G}, p, g), \kappa, \mathcal{M}, \mathcal{S})$, $pk = X$ and $\sigma = (R, s)$. Compute $c = H.\text{Eval}(\kappa, (R, m))$. Accept if and only if:

$$g^s = R \cdot X^c$$

Lemma 10.1 (Correctness of Schnorr Signatures). The Schnorr signature scheme $\text{SchnorrSig}[\text{GrGen}, H]$ is correct.

Proof. Let $\sigma = (R, s)$ be a signature generated by $\text{Sign}(pp, sk, m)$ where $sk = x$ and $pk = X = g^x$. By the signing algorithm, we have $R = g^r$ for some $r \leftarrow \mathbb{Z}_p$, $c = H.\text{Eval}(\kappa, (R, m))$, and $s = r + c \cdot x \bmod p$. The verification equation holds because:

$$g^s = g^{r+cx} = g^r \cdot g^{cx} = R \cdot (g^x)^c = R \cdot X^c$$

Therefore, $\text{Verify}(pp, pk, m, \sigma) = 1$ for all honestly generated signatures. $\square$

Remark 10.1. In the security analysis that follows, we work in the Random Oracle Model where the hash function $H.\text{Eval}(\kappa, \cdot)$ is modeled as a truly random function. In the ROM analysis, we write $H(\cdot)$ instead of $H.\text{Eval}(\kappa, \cdot)$, with the understanding that H represents a random oracle.

Game $\text{EUF-CMA}_{\text{SchnorrSig}}^A(\lambda)$	$\mathcal{B}((\mathbb{G}, p, g), X; \rho)$	$\mathcal{C}^H((\mathbb{G}, p, g), X; \rho)$
$(\mathbb{G}, p, g) := \text{GrGen}(1^\lambda)$	$pp := ((\mathbb{G}, p, g), \{0, 1\}^*, \mathbb{G} \times \mathbb{Z}_p)$	$pp := ((\mathbb{G}, p, g), \{0, 1\}^*, \mathbb{G} \times \mathbb{Z}_p)$
$x \leftarrow \$ \mathbb{Z}_p$	$T := \emptyset$	$T := \emptyset$
$X := g^x$	$\mathcal{Q} := \emptyset$	$\mathcal{Q} := \emptyset$
$pp := ((\mathbb{G}, p, g), \{0, 1\}^*, \mathbb{G} \times \mathbb{Z}_p)$	$(m^*, \sigma^*) := \mathcal{A}^{\text{SIGN}_{\mathcal{B}}, \text{H}_{\mathcal{B}}}(pp, X; \rho)$	$(m^*, \sigma^*) := \mathcal{A}^{\text{SIGN}_{\mathcal{C}}, \text{H}_{\mathcal{C}}}(pp, X; \rho)$
$T := \emptyset$	$(R^*, s^*) := \sigma^*$	$(R^*, s^*) := \sigma^*$
$\mathcal{Q} := \emptyset$	$c^* := H_{\mathcal{B}}((R^*, m^*))$	$c^* := H_{\mathcal{C}}((R^*, m^*))$
$(m^*, \sigma^*) := \mathcal{A}^{\text{SIGN}, \text{H}_{\text{UF}}}(pp, X)$	assert $m^* \notin \mathcal{Q}$	assert $m^* \notin \mathcal{Q}$
$(R^*, s^*) := \sigma^*$	assert $g^{s^*} = R^* \cdot X^{c^*}$	assert $g^{s^*} = R^* \cdot X^{c^*}$
$c^* := H_{\text{UF}}(R^*, m^*)$	return $((R^*, m^*), s^*)$	return $((R^*, m^*), s^*)$
return $m^* \notin \mathcal{Q} \wedge g^{s^*} = R^* \cdot X^{c^*}$		
Oracle $\text{SIGN}(m)$	Oracle $\text{SIGN}_{\mathcal{B}}(m)$	Oracle $\text{SIGN}_{\mathcal{C}}(m)$
$r \leftarrow \$ \mathbb{Z}_p$	$s \leftarrow \$ \mathbb{Z}_p$	$s \leftarrow \$ \mathbb{Z}_p$
$R := g^r$	$c \leftarrow \$ \mathbb{Z}_p$	$c \leftarrow \$ \mathbb{Z}_p$
$c := H_{\text{UF}}(R, m)$	$R := g^s \cdot X^{-c}$	$R := g^s \cdot X^{-c}$
$s := r + c \cdot x \bmod p$	assert $T[R, m] = \perp$	assert $T[R, m] = \perp$
$\mathcal{Q} := \mathcal{Q} \cup \{m\}$	$T[R, m] := c$	$T[R, m] := c$
return (R, s)	$\mathcal{Q} := \mathcal{Q} \cup \{m\}$	$\mathcal{Q} := \mathcal{Q} \cup \{m\}$
	return (R, s)	return (R, s)
Oracle $H_{\text{UF}}(y)$	Oracle $H_{\mathcal{B}}(y)$	Oracle $H_{\mathcal{C}}(y)$
if $T[y] = \perp$ then	if $T[y] = \perp$ then	if $T[y] = \perp$ then
$T[y] \leftarrow \$ \mathbb{Z}_p$	$T[y] \leftarrow \$ \mathbb{Z}_p$	$T[y] := H(y)$
return $T[y]$	return $T[y]$	return $T[y]$

Fig. 10.2. Security reduction for Schnorr signatures: EUF-CMA game (left), algorithm $\mathcal{B}$ (middle), and algorithm $\mathcal{C}$ (right). Changes from the previous game are marked with $\blacksquare$.

Lemma 10.2. Let $\mathcal{A}$ be an adversary against the EUF-CMA security of $\text{SchnorrSig}[\text{GrGen}, H]$ in the random oracle model for H making at most q_s queries to SIGN and at most q_h queries to H .

Let InpGen be the algorithm which on input 1^λ runs $(\mathbb{G}, p, g) := \text{GrGen}(1^\lambda)$, draws $X \leftarrow \$ \mathbb{G}$, and returns $((\mathbb{G}, p, g), X)$.

Consider algorithm $\mathcal{C}$ defined in Figure 10.2 that takes as input $((\mathbb{G}, p, g), X) := \text{InpGen}(1^\lambda)$ and has access to a random oracle H . Then $\mathcal{C}$ makes at most $q_h + 1$ queries to H , and satisfies

$$\left| \text{Adv}_{\mathcal{C}, \text{InpGen}}^{\text{single}}(\lambda) - \text{Adv}_{\mathcal{A}, \text{SchnorrSig}[\text{GrGen}, H]}^{\text{euf-cma}}(\lambda) \right| \leq \frac{2q_s(q_s + q_h)}{2^\lambda}$$

with $\text{Adv}_{\mathcal{C}, \text{InpGen}}^{\text{single}}(\lambda)$ as defined in the forking lemma (Lemma 9.1).

Moreover, when $\mathcal{C}$ returns a non- $\perp$ output (z, aux) , then $z = (R^*, m^*)$, $\text{aux} = s^*$, and

$$g^{s^*} = R^* \cdot X^{H(R^*, m^*)}$$

The proof is left as an exercise (see [Exercise 10.3](#)).

Theorem 10.1 (Schnorr signatures are EUF-CMA secure). *Let GrGen be a group generation algorithm for which the discrete logarithm problem is hard and let H be a hash function with output space $\mathbb{Z}_p$. Then the Schnorr signature scheme SchnorrSig[GrGen, H] is EUF-CMA secure in the random oracle model for H. More precisely, for any p.p.t. adversary $\mathcal{A}$ against the EUF-CMA security of SchnorrSig[GrGen, H] making at most q_s queries to the signing oracle and at most q_h queries to the random oracle H, there exists a p.p.t. adversary $\mathcal{D}$ against the discrete logarithm problem such that*

$$\text{Adv}_{\mathcal{A}, \text{SchnorrSig}[\text{GrGen}, \text{H}]}^{\text{euf-cma}}(\lambda) \leq \frac{2q_s(q_s + q_h)}{2^\lambda} + \sqrt{(q_h + 1) \left(\text{Adv}_{\mathcal{D}, \text{GrGen}}^{\text{dlog}}(\lambda) + \frac{1}{2^\lambda} \right)}.$$

The proof is left as an exercise (see [Exercise 10.4](#)).

10.4 Exercises

Exercise 10.1. The EUF-CMA security definition in [Definition 10.3](#) only requires the forged message to be new. An alternative notion called *strong EUF-CMA* would allow the adversary to win if they produce a different signature for a message even if it was previously queried. Define a strong EUF-CMA security game that captures this intuition.

Exercise 10.2. In the Generic Group Model (GGM) [[Sho97](#)], the advantage of solving the discrete logarithm problem with q group operations is bounded by $\text{Adv}_{\mathcal{A}, \text{GrGen}}^{\text{dlog}}(\lambda) \leq \frac{q^2}{2p}$ where p is the prime group order.

Consider Schnorr signatures over a 256-bit elliptic curve group (e.g., secp256k1 where $p > 2^{255}$). Suppose an adversary can make:

- $q_s = 2^{10}$ signing queries
- $q_h = 2^{85}$ hash queries
- $q = 2^{85}$ group operations

Calculate the concrete EUF-CMA advantage using the bound from [Theorem 10.1](#) with $\lambda = 256$, and compare it to the discrete logarithm advantage $\text{Adv}_{\mathcal{A}, \text{GrGen}}^{\text{dlog}}(\lambda)$ in the GGM.

Exercise 10.3. Prove [Lemma 10.2](#) using the games in [Figure 10.2](#).

Exercise 10.4. Prove [Theorem 10.1](#). Use [Lemma 10.2](#) and the forking lemma ([Lemma 9.1](#)).

Exercise 10.5 (Optional). Study the formalization of adaptor signatures in Gerhart, Schröder, Soni, and Thyagarajan [[Ger+24](#)]. Explain what gaps existed in previous formalizations and how this work addresses them.

11 Signatures with Key Tweaking

Key tweaking is fundamental in hierarchical deterministic (HD) Bitcoin wallets [[BIP32](#)] and Taproot commitments [[BIP341](#)], enabling the derivation of multiple keys from a single master key and committing to additional data in public keys.

11.1 Syntax

Definition 11.1 (Tweaking Scheme for Signatures). *A tweaking scheme TwSch for a signature scheme Sig is a tuple $(\mathcal{T}, \text{TwSK}, \text{TwPK})$ where:*

- $\mathcal{T}$ is a set of allowed tweaks, parameterized by the public parameters pp output by Setup.
- $\text{TwSK}(sk, t) \rightarrow \tilde{sk}$ is a deterministic polynomial-time algorithm that takes a secret key sk and a tweak $t \in \mathcal{T}$ as input and outputs a tweaked secret key $\tilde{sk}$.
- $\text{TwPK}(pk, t) \rightarrow \tilde{pk}$ is a deterministic polynomial-time algorithm that takes a public key pk and a tweak $t \in \mathcal{T}$ as input and outputs a tweaked public key $\tilde{pk}$.

11.2 Key Tweaking for Schnorr Signatures

Definition 11.2 (SchnorrTS). The tweaking scheme SchnorrTS for Schnorr signatures is defined as follows:

- The tweak set is $\mathcal{T} := \mathbb{Z}_p \times \mathbb{Z}_p^*$ where $\mathbb{Z}_p^* = \mathbb{Z}_p \setminus \{0\}$.
- For $(\alpha, \beta) \in \mathcal{T}$, the secret key tweaking algorithm is:

$$\text{TwSK}(x, (\alpha, \beta)) := \alpha + \beta \cdot x \bmod p$$

for every $x \in \mathbb{Z}_p$.

- For $(\alpha, \beta) \in \mathcal{T}$, the public key tweaking algorithm is:

$$\text{TwPK}(X, (\alpha, \beta)) := g^\alpha \cdot X^\beta$$

for every $X \in \mathbb{G}$.

Definition 11.3 (Schnorr Signature with Key Prefixing). The Schnorr signature scheme with key prefixing SchnorrSigKP[GrGen, H] is identical to SchnorrSig[GrGen, H] except that the hash is computed as:

$$c = \text{H.Eval}(\kappa, (R, X, m))$$

instead of $c = \text{H.Eval}(\kappa, (R, m))$, where X is the public key.

11.3 Security with Tweaked Keys

The security notion of *existential unforgeability under chosen message attack with tweaked keys* (EUF-CMA-TK) extends the standard EUF-CMA notion to capture security when an adversary can obtain signatures under tweaked keys. Informally, the adversary is given a public key and can request signatures on messages of their choice, but now with an additional capability: for each signing query, they can specify a tweak in addition to the message. The signing oracle will return a signature that verifies under the tweaked public key (i.e., a signature created with the correspondingly tweaked secret key). The adversary wins by outputting (m^*, t^*, σ^*) such that the signature verifies under the tweaked public key for t^* , and the pair (m^*, t^*) was never queried to the signing oracle.

The formal definition of EUF-CMA-TK is left as an exercise (see [Exercise 11.3](#)).

Theorem 11.1 (SchnorrSigKP with SchnorrTS is EUF-CMA-TK secure). Let GrGen be a group generation algorithm for which the discrete logarithm problem is hard and let H be a hash function with output space $\mathbb{Z}_p$. Then the Schnorr signature scheme with key prefixing SchnorrSigKP[GrGen, H] with tweaking scheme SchnorrTS is EUF-CMA-TK secure in the random oracle model for H.

The proof is left as an exercise (see [Exercise 11.6](#)).

11.4 Exercises

Exercise 11.1. Define correctness for a signature scheme with tweaking. Specifically, modify standard signature scheme correctness ([Definition 10.2](#)) to ensure that signatures created with a tweaked secret key verify correctly under the corresponding tweaked public key.

Exercise 11.2 (Optional). Argue that the tweaking scheme SchnorrTS ([Definition 11.2](#)) is correct with the Schnorr signature scheme ([Definition 10.4](#)). That is, show that signatures created with a tweaked secret key verify correctly under the corresponding tweaked public key.

Exercise 11.3. Formalize the EUF-CMA-TK security notion by adapting EUF-CMA security ([Definition 10.3](#)). Your definition should capture the informal description given in [subsection 11.3](#).

Exercise 11.4. Give a p.p.t. adversary that breaks the EUF-CMA-TK security of SchnorrSig with the SchnorrTS tweaking scheme. Hints:

- SchnorrSig does not use key prefixing.
- You only need to use the signing oracle once.
- Pick a tweak t and look at the Schnorr verification equation for a signature (R, s) under public key X . Compute s' by adding a value to s such that (R, s') is a valid signature for $\text{TwPK}(X, t)$.

Exercise 11.5. Explain why we cannot directly apply the EUF-CMA security proof of SchnorrSig to prove EUF-CMA-TK security for SchnorrSig with SchnorrTS tweaking.

Hint: Examine what Lemma 10.2 guarantees about $\mathcal{C}$'s output and why this guarantee holds. Then consider whether a similar guarantee could be established for a reduction $\mathcal{C}'$ that uses an adversary $\mathcal{A}$ breaking EUF-CMA-TK for SchnorrSig with SchnorrTS.

Exercise 11.6. Prove the following lemma, which is an adaptation of Lemma 10.2 for the EUF-CMA-TK security of SchnorrSigKP with SchnorrTS.

Lemma 11.1. *Let $\mathcal{A}$ be an adversary against the EUF-CMA-TK security of SchnorrSigKP with SchnorrTS[GrGen, H] in the random oracle model for H making at most q_s queries to SIGN and at most q_h queries to H.*

Let InpGen be the algorithm which on input 1^λ runs $(\mathbb{G}, p, g) := \text{GrGen}(1^\lambda)$, draws $X \leftarrow \mathbb{G}$, and returns $((\mathbb{G}, p, g), X)$.

There exists algorithm $\mathcal{C}$ that takes as input $((\mathbb{G}, p, g), X) := \text{InpGen}(1^\lambda)$, has access to a random oracle H, makes at most $q_h + 1$ queries to H, and satisfies

$$\left| \text{Adv}_{\mathcal{C}, \text{InpGen}}^{\text{single}}(\lambda) - \text{Adv}_{\mathcal{A}, \text{SchnorrSigKP}, \text{SchnorrTS}}^{\text{euf-cma-tk}}(\lambda) \right| \leq \frac{q_s(q_s + q_h)}{2^\lambda}$$

with $\text{Adv}_{\mathcal{C}, \text{InpGen}}^{\text{single}}(\lambda)$ as defined in the forking lemma (Lemma 9.1).

Moreover, when $\mathcal{C}$ returns a non- $\perp$ output (z, aux) , then $z = (R^, \text{TwPK}(X, t^*), m^*, t^*)$, $\text{aux} = s^*$, and*

$$g^{s^*} = R^* \cdot \text{TwPK}(X, t^*)^{H(R^*, \text{TwPK}(X, t^*), m^*)}$$

Adapt the proof of Lemma 10.2 to prove this lemma.

Note: Applying the forking lemma to $\mathcal{C}$ would allow extracting the discrete logarithm via $g^{s_1 - s_2} = (g^\alpha \cdot X^\beta)^{h_1 - h_2}$ where $(\alpha, \beta) = t^*$, thus establishing the EUF-CMA-TK security of SchnorrSigKP with SchnorrTS.

Exercise 11.7 (Optional). Read about signatures with re-randomizable keys in Section 3 of Fleischhacker et al. [Fle+16], which presents a notion closely related to key tweaking. Compare their security model with the EUF-CMA-TK model discussed in this section and discuss the practical impact. How can they prove security for Schnorr signatures with re-randomized keys without requiring public key prefixing?

12 Interactive Aggregate Signatures

An interactive aggregate signature (IAS) scheme allows n signers, each with their own key pair (sk_i, pk_i) and message m_i , to jointly produce a single short signature that proves m_i was signed under pk_i for every $i \in \{1, \dots, n\}$.

The following exercises are based on the DahLIAS paper [NRS25], which presents a discrete logarithm-based IAS scheme with constant-size signatures.

12.1 Exercises

Exercise 12.1. Read the abstract and the main security theorem (Theorem 2) of DahLIAS. What does the security theorem tell us about the relationship between breaking DahLIAS and solving the underlying computational problems?

Exercise 12.2. Read the technical introduction (Section 1.3). Why does the straightforward adaptation of the MuSig2 two-round technique described in the text not yield a secure IAS scheme?

Exercise 12.3. Read about IAS syntax and security (Sections 3.1 and 3.3). What is the difference between EUF-CMA and co-EUF-CMA security for IAS?

Exercise 12.4. What are the applications where strong binding (Section 6) matters? Explain why this property is important in these contexts.

Exercise 12.5 (Optional). Read the proof sketch (Section 5.2). Outline the main proof technique and explain how the security reduction works.

A Notation

A.1 Sets and Numbers

- $\mathbb{N}$ denotes the set of natural numbers $\{0, 1, 2, \dots\}$.
- $\mathbb{Z}$ denotes the set of integers.
- $\mathbb{Z}_p$ denotes the set of integers modulo p , i.e., $\{0, 1, \dots, p-1\}$.
- $\mathbb{Z}_p^*$ denotes the set of integers modulo p excluding zero, i.e., $\{1, 2, \dots, p-1\}$.
- $[n]$ denotes the set $\{1, 2, \dots, n\}$ for a positive integer n .
- $[a, b]$ denotes the set $\{a, a+1, \dots, b\}$ for integers $a \leq b$.
- $\{0, 1\}^n$ denotes the set of all binary strings of length n .
- $\{0, 1\}^*$ denotes the set of all binary strings of finite length.

A.2 Probability and Sampling

- $x \leftarrow \$ S$ denotes sampling x uniformly at random from the finite set S .
- $x := A(y)$ denotes running algorithm A on input y and assigning the output to x .
- $\Pr[E]$ denotes the probability of event E .
- $:=$ denotes a definition, i.e., the left side is defined to equal the right side.

A.3 Cryptographic Notation

- λ denotes the security parameter.
- 1^λ denotes the security parameter in unary representation.
- $\text{negl}(\lambda)$ denotes a negligible function in the security parameter.
- p.p.t. stands for probabilistic polynomial-time.

A.4 Groups and Fields

- $(\mathbb{G}, p, g)$ denotes group parameters, where $\mathbb{G}$ denotes a cyclic group, g denotes the generator and p denotes the order of a group.
- $1_{\mathbb{G}}$ denotes the identity element of group $\mathbb{G}$.

A.5 Logical Operators

- $\wedge$ denotes logical AND.
- $\vee$ denotes logical OR.
- $\neg$ denotes logical NOT.
- $\oplus$ denotes bitwise XOR.

A.6 Other Notation

- $\|$ denotes concatenation (e.g., $x\|y$ is the concatenation of x and y).
- $|S|$ denotes the cardinality (size) of set S .
- $\perp$ denotes an error or undefined value.
- $\emptyset$ denotes the empty set.
- $\log_g(h)$ denotes the discrete logarithm of h with respect to base g .
- $O(\cdot)$ denotes asymptotic upper bound (big-O notation).

B Appendix: Proofs of Negligible Function Properties

This appendix contains the proofs of properties (2)-(4) from Lemma 1.1.

Proof of Lemma 1.1(2). Let f_1, f_2 be negligible functions. We need to show that $f_1 + f_2$ is negligible.

Let $c > 0$ be arbitrary. We must find N such that $(f_1 + f_2)(\lambda) < \lambda^{-c}$ for all $\lambda > N$.

Since f_1 and f_2 are negligible, for the constant $c + 1 > 0$, there exist $N_1, N_2 \in \mathbb{N}$ such that:

$$\forall \lambda > N_1 : f_1(\lambda) < \lambda^{-(c+1)} \quad \text{and} \quad \forall \lambda > N_2 : f_2(\lambda) < \lambda^{-(c+1)}$$

Let $N = \max(N_1, N_2, 2)$. Then for all $\lambda > N$:

$$f_1(\lambda) + f_2(\lambda) < \lambda^{-(c+1)} + \lambda^{-(c+1)} = 2 \cdot \lambda^{-(c+1)} = \frac{2}{\lambda} \cdot \lambda^{-c} \leq \lambda^{-c}$$

where the last inequality holds because $\frac{2}{\lambda} \leq 1$ when $\lambda \geq 2$, which is guaranteed by our choice of N . $\square$

Proof of Lemma 1.1(3). Let f be a negligible function with $f(\lambda) \geq 0$ and let $k > 0$ be a constant. We need to show that $f + k$ is not negligible.

Assume for contradiction that $f + k$ is negligible. Then for the constant $c = 1$, there exists $N \in \mathbb{N}$ such that for all $\lambda > N$:

$$f(\lambda) + k < \lambda^{-1}$$

Since $f(\lambda) \geq 0$, we have $f(\lambda) + k \geq k$ for all λ .

Choose $\lambda_0 = \max(N + 1, \lceil \frac{2}{k} \rceil)$. Then $\lambda_0 > N$ and $\lambda_0^{-1} \leq \frac{k}{2}$.

By our assumption, we must have:

$$k \leq f(\lambda_0) + k < \lambda_0^{-1} \leq \frac{k}{2}$$

This gives us $k < \frac{k}{2}$, which is a contradiction.

Therefore, $f + k$ is not negligible. $\square$

Proof of Lemma 1.1(4). Let f be a non-negligible function and g be a negligible function. Assume for contradiction that $f - g$ is negligible. Then by property (2), $(f - g) + g = f$ would be negligible (as the sum of two negligible functions). This contradicts the assumption that f is non-negligible. $\square$

C Appendix: Notation

Acknowledgements

We thank Nadav Kohen and Pieter Wuille for valuable feedback and discussions. We also thank Claude for assistance with revisions and exercises.

Changelog

2025-08-24

– First public version.

References

- [BCY20] Foteini Baldimtsi, Ran Canetti, and Sophia Yakoubov. “Universally Composable Accumulators”. In: *CT-RSA 2020*. DOI: [10.1007/978-3-030-40186-3_27](https://doi.org/10.1007/978-3-030-40186-3_27).
- [BDL19] Mihir Bellare, Wei Dai, and Lucy Li. “The Local Forking Lemma and Its Application to Deterministic Encryption”. In: *ASIACRYPT 2019, Part III*. DOI: [10.1007/978-3-030-34618-8_21](https://doi.org/10.1007/978-3-030-34618-8_21).
- [BIP32] Pieter Wuille. *Hierarchical Deterministic Wallets*. Bitcoin Improvement Proposal 32. See <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>. 2012.
- [BIP341] Pieter Wuille, Jonas Nick, and Anthony Towns. *Taproot: SegWit version 1 spending rules*. Bitcoin Improvement Proposal 341. See <https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki>. 2020.
- [BIP352] josibake and Ruben Somsen. *Silent Payments*. See <https://github.com/bitcoin/bips/blob/master/bip-0352.mediawiki>. 2023.
- [BL13] Daniel J. Bernstein and Tanja Lange. “Non-uniform Cracks in the Concrete: The Power of Free Precomputation”. In: *ASIACRYPT 2013, Part II*. DOI: [10.1007/978-3-642-42045-0_17](https://doi.org/10.1007/978-3-642-42045-0_17).
- [BN06] Mihir Bellare and Gregory Neven. “Multi-signatures in the plain public-Key model and a general forking lemma”. In: *ACM CCS 2006*. DOI: [10.1145/1180405.1180453](https://doi.org/10.1145/1180405.1180453).
- [BR93] Mihir Bellare and Phillip Rogaway. “Random Oracles are Practical: A Paradigm for Designing Efficient Protocols”. In: *ACM CCS 93*. DOI: [10.1145/168588.168596](https://doi.org/10.1145/168588.168596).
- [CGH98] Ran Canetti, Oded Goldreich, and Shai Halevi. “The Random Oracle Methodology, Revisited (Preliminary Version)”. In: *30th ACM STOC*. DOI: [10.1145/276698.276741](https://doi.org/10.1145/276698.276741).
- [CLW24] Cas Cremers, Julian Loss, and Benedikt Wagner. “A Holistic Security Analysis of Monero Transactions”. In: *EUROCRYPT 2024, Part III*. DOI: [10.1007/978-3-031-58734-4_5](https://doi.org/10.1007/978-3-031-58734-4_5).
- [Dri+19] Manu Drijvers, Kasra Edalatnejad, Bryan Ford, Eike Kiltz, Julian Loss, Gregory Neven, and Igors Stepanovs. “On the Security of Two-Round Multi-Signatures”. In: *2019 IEEE Symposium on Security and Privacy*. DOI: [10.1109/SP.2019.00050](https://doi.org/10.1109/SP.2019.00050).
- [Fle+16] Nils Fleischhacker, Johannes Krupp, Giulio Malavolta, Jonas Schneider, Dominique Schröder, and Mark Simkin. “Efficient Unlinkable Sanitizable Signatures from Signatures with Re-randomizable Keys”. In: *PKC 2016, Part I*. DOI: [10.1007/978-3-662-49384-7_12](https://doi.org/10.1007/978-3-662-49384-7_12).
- [Ger+24] Paul Gerhart, Dominique Schröder, Pratik Soni, and Sri Aravinda Krishnan Thyagarajan. “Foundations of Adaptor Signatures”. In: *EUROCRYPT 2024, Part II*. DOI: [10.1007/978-3-031-58723-8_6](https://doi.org/10.1007/978-3-031-58723-8_6).
- [HRS16] Andreas Hülsing, Joost Rijneveld, and Fang Song. “Mitigating Multi-target Attacks in Hash-Based Signatures”. In: *PKC 2016, Part I*. DOI: [10.1007/978-3-662-49384-7_15](https://doi.org/10.1007/978-3-662-49384-7_15).
- [Hül13] Andreas Hülsing. “W-OTS+ - Shorter Signatures for Hash-Based Signature Schemes”. In: *AFRICACRYPT 13*. DOI: [10.1007/978-3-642-38553-7_10](https://doi.org/10.1007/978-3-642-38553-7_10).
- [KM15] Neal Koblitz and Alfred J. Menezes. “The random oracle model: a twenty-year retrospective”. In: *DCC 77.2-3 (2015)*. DOI: [10.1007/s10623-015-0094-2](https://doi.org/10.1007/s10623-015-0094-2).
- [Mau05] Ueli M. Maurer. “Abstract Models of Computation in Cryptography (Invited Paper)”. In: *10th IMA International Conference on Cryptography and Coding*. DOI: [10.1007/11586821_1](https://doi.org/10.1007/11586821_1).
- [Max+19] Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille. “Simple Schnorr multi-signatures with applications to Bitcoin”. In: *DCC 87.9 (2019)*. DOI: [10.1007/s10623-019-00608-x](https://doi.org/10.1007/s10623-019-00608-x).
- [Nao03] Moni Naor. “On Cryptographic Assumptions and Challenges (Invited Talk)”. In: *CRYPTO 2003*. DOI: [10.1007/978-3-540-45146-4_6](https://doi.org/10.1007/978-3-540-45146-4_6).
- [NEL25] Jonas Nick, Liam Eagen, and Robin Linus. “Shielded CSV: Private and Efficient Client-Side Validation”. In: *Cryptology ePrint Archive (2025)*.
- [NRS25] Jonas Nick, Tim Ruffing, and Yannick Seurin. “DahLIAS: Discrete Logarithm-Based Interactive Aggregate Signatures”. In: *Cryptology ePrint Archive (2025)*.
- [Rog06] Phillip Rogaway. “Formalizing Human Ignorance”. In: *Progress in Cryptology - VI-ETCRYPT 06*. DOI: [10.1007/11958239_14](https://doi.org/10.1007/11958239_14).

- [Sho97] Victor Shoup. “Lower Bounds for Discrete Logarithms and Related Problems”. In: *EUROCRYPT’97*. DOI: [10.1007/3-540-69053-0_18](https://doi.org/10.1007/3-540-69053-0_18).